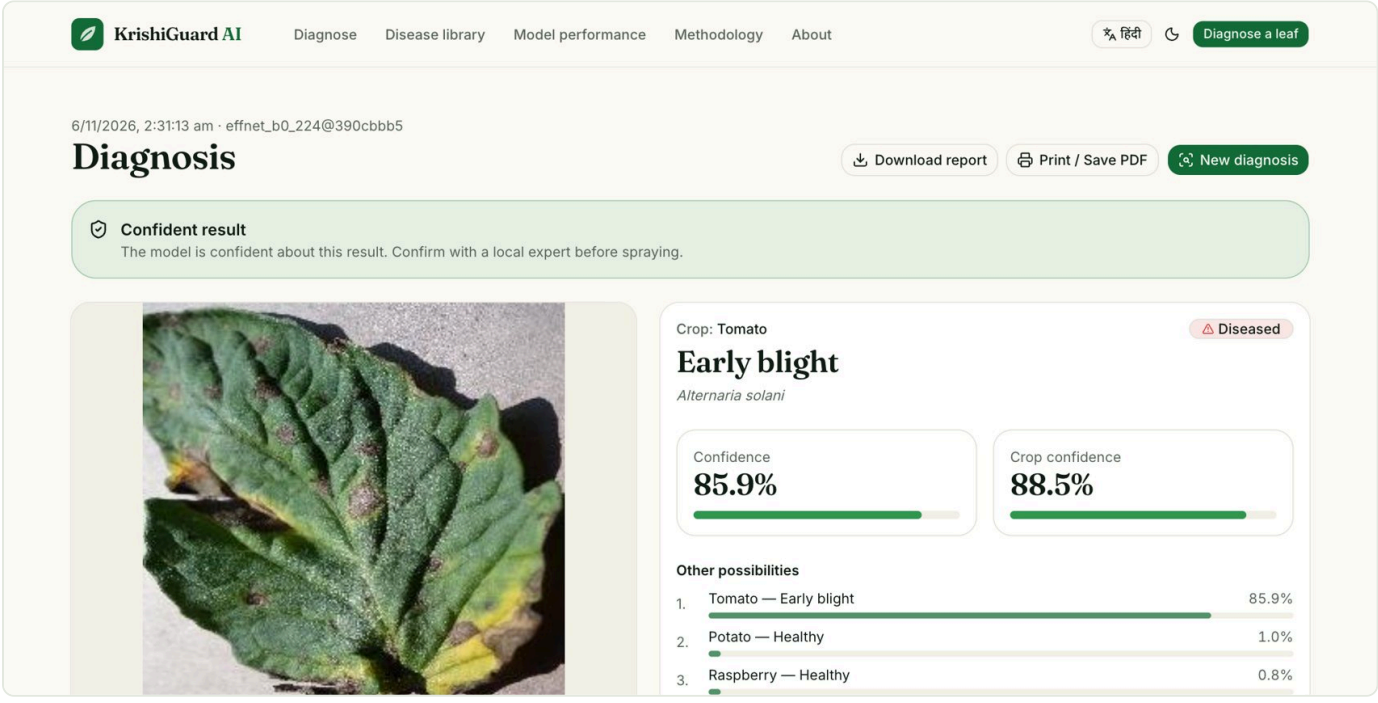
A PROJECT REPORT ON

KrishiGuard AI

Intelligent Crop Disease Detection and Advisory System using Deep Learning (Transfer Learning with EfficientNet-B0)

Submitted as a Deep Learning project

Domain	Computer Vision · Deep Learning · Agriculture
Model	EfficientNet-B0 (ImageNet-pretrained, fine-tuned in PyTorch); served with ONNX Runtime
Dataset	PlantVillage — 54,305 images, 38 classes, 14 crops; external field datasets PlantDoc and Tomato (Pakistan)
Headline result	**99.56%** accuracy and **99.49%** macro-F1 on 7,751 held-out test images
Application	Responsive bilingual (English / Hindi) web application — krishiguard-ai-five.vercel.app
Report date	November 2026



Diagnosis result page of the application (real model output)

Abstract

Plant diseases cause large yield losses, and farmers often lack quick access to an expert. **KrishiGuard AI** is a responsive web application in which a farmer uploads, photographs or pastes an image of a crop leaf and receives the **crop**, the **disease (or healthy)**, a **calibrated confidence**, a **heat-map explanation** and **advisory in English and Hindi** (symptoms, causes, prevention, organic and chemical options with cited agricultural sources).

An ImageNet-pretrained **EfficientNet-B0** convolutional neural network was fine-tuned with transfer learning on the PlantVillage dataset (54,305 images, 38 classes, 14 crops) using a **leakage-free, group-level split**. On 7,751 held-out test images it reaches **99.56% accuracy**, 99.46% macro precision, 99.52% macro recall and **99.49% macro-F1**. Testing on two independent real-field datasets (PlantDoc and a tomato field dataset from Pakistan) shows the well-known *lab-to-field gap* (31.1% and 26.5% accuracy); the system therefore uses temperature scaling and validation-selected thresholds to label results as *confident*, *uncertain* or *not recognised*, and shows treatment advice only for confident results.

The model is served with ONNX Runtime from a FastAPI backend and a Next.js frontend, both deployed free on Vercel. The project includes 44 backend tests, 6 component tests and 16 end-to-end browser tests (desktop and mobile), all passing.

Keywords: plant disease detection, transfer learning, EfficientNet, PlantVillage, calibration, Grad-CAM, ONNX, FastAPI, Next.js, Hindi advisory.

Contents

Abstract

Chapter 1. Introduction

Chapter 2. Literature Review

Chapter 3. Development Environment and Requirements

Chapter 4. System Architecture and Technology Stack

Chapter 5. Datasets and Exploratory Data Analysis

Chapter 6. Data Preprocessing and Leakage-free Splitting

Chapter 7. Model Architecture and Training

Chapter 8. Confidence Calibration and Uncertainty Handling

Chapter 9. Results and Evaluation

Chapter 10. Explainability (Grad-CAM / CAM)

Chapter 11. Bilingual Advisory Engine

Chapter 12. Web Application

Chapter 13. Backend API and Model Serving

Chapter 14. Testing and Quality Assurance

Chapter 15. Deployment

Chapter 16. Limitations and Future Scope

Chapter 17. Conclusion

References

Appendix A — Per-class results

Appendix B — Dataset split per class

List of Figures

Figure 1. Number of images per PlantVillage class.

Figure 2. Random PlantVillage samples: single leaves on a uniform background (lab-style).

Figure 3. PlantDoc samples: real backgrounds, multiple leaves, varied light.

Figure 4. Tomato field samples (Pakistan), smartphone photos.

Figure 5. Per-image mean colour by dataset — the field datasets differ clearly from PlantVillage (domain shift).

Figure 6. Per-class distribution of the leakage-free split.

Figure 7. Training curves: loss (left) and accuracy / macro-F1 (right). Validation accuracy is near its ceiling after epoch 1; field (PlantDoc) performance stays low throughout.

Figure 8. Risk–coverage: accuracy on the images the model answers as the confidence bar is raised.

Figure 9. Reliability diagram (PlantDoc test) before and after temperature scaling.

Figure 10. Row-normalised confusion matrix on the PlantVillage test set (38 classes).

Figure 11. Internal (lab) vs external (field) macro-F1 of the trained model.

Figure 12. Confusion matrix — PlantDoc test (classes present).

Figure 13. Confusion matrix — Tomato field test.

Figure 14. The most confident mistakes on the PlantVillage test set (true vs predicted, calibrated confidence).

Figure 15. The most confident mistakes on tomato field photos.

Figure 16. Model input, heat-map and overlay for lab (top four rows) and field (bottom four rows) images. On lab images the model focuses on the leaf and lesions; on field photos attention is scattered and predictions are wrong with low confidence.

Figure 17. Treatment tab in the application with organic and chemical options and the label-safety notice.

Figure 18. Landing page — live metrics are loaded from the evaluation results.

Figure 19. Diagnose page: upload, camera and paste options, supported formats.

Figure 20. Selected photo with preview, replace / camera / paste / remove actions.

Figure 21. Results: status banner, crop and disease, calibrated confidence, top-3 candidates, heat-map and advisory.

Figure 22. The same result after switching to Hindi.

Figure 23. Heat-map viewer: model input, heat-map and overlay (also a slider compare mode).

Figure 24. A field photo outside the training distribution is reported as not recognised; no advice is shown.

Figure 25. Disease library with search and crop filters.
Figure 26. Disease detail sheet with cited advisory.
Figure 27. Model performance dashboard (top part) — KPIs, lab-vs-field chart, risk–coverage and model summary; every number comes from the real evaluation.
Figure 28. Mobile — landing.
Figure 29. Mobile — diagnose.
Figure 30. Mobile — result.

CHAPTER 1

Introduction

1.1 Problem statement

Farmers should be able to photograph a crop leaf and immediately learn whether it is diseased, which disease it is likely to be, how sure the system is, and what preventive and treatment options exist — in a language they understand. The solution must run in any browser (desktop, tablet, phone) without installing an app.

1.2 Objectives

1. Identify the crop and disease (or healthy state) from a single leaf photo.
2. Report a confidence score and clearly flag uncertain or unrecognised photos.
3. Explain the prediction visually with a heat-map.
4. Provide symptoms, causes, prevention, organic and chemical options in **English and Hindi**, with sources.
5. Evaluate honestly on an untouched test set and on independent real-field images.
6. Deliver a professional, responsive web application with a real performance dashboard.

1.3 Scope

38 classes over 14 crops: apple, blueberry, cherry, corn (maize), grape, orange, peach, bell pepper, potato, raspberry, soybean, squash, strawberry and tomato (26 diseases + 12 healthy classes). The system is a decision-support tool and not a substitute for an agricultural expert.

1.4 Organisation of the report

The following chapters review related work, describe the development environment, the system architecture, the datasets and their preparation, the model and every training experiment performed, calibration, results, explainability, the advisory engine, the web application and API, testing and deployment, and finally the limitations and conclusions.

CHAPTER 2

Literature Review

2.1 Deep learning for plant disease classification

Mohanty, Hughes and Salathé (2016) introduced large-scale deep learning for leaf disease recognition on the PlantVillage dataset (54,306 images, 38 classes) and reported up to 99.35 % accuracy with GoogLeNet trained by transfer learning. They also noted that accuracy fell sharply on images taken under conditions different from the training data — the first documented sign of the lab-to-field gap that is examined again in this project.

2.2 Real-world (field) datasets

Singh et al. (2020) released **PlantDoc**, a dataset of 2,598 images collected from the internet with natural backgrounds and multiple leaves per image. Models trained only on PlantVillage performed poorly on PlantDoc, which motivated using PlantDoc and an independent tomato field dataset as *external* test sets in this work.

2.3 Efficient convolutional networks and transfer learning

Transfer learning reuses features learned on ImageNet and fine-tunes them on a smaller target dataset, giving high accuracy with modest data and compute. **EfficientNet** (Tan and Le, 2019) scales network depth, width and input resolution jointly; its smallest member, EfficientNet-B0 (≈ 5.3 M parameters with the ImageNet head, 4.1 M with a 38-class head), offers a good accuracy–cost trade-off for CPU deployment, which is why it was chosen here.

2.4 Calibration and explainability

Guo et al. (2017) showed that modern neural networks are over-confident and that a single **temperature** parameter fitted on validation data substantially improves calibration. Selvaraju et al. (2017) proposed **Grad-CAM**, which produces class-specific heat-maps from gradients of the last convolutional layer; for networks ending in global average pooling it reduces to the Class Activation Map (CAM) of Zhou et al. (2016). Both ideas are used to make the predictions of this system more trustworthy.

2.5 Gap addressed by this project

Many student and research prototypes report only PlantVillage accuracy with random splits. This project adds (1) a leakage-free group split, (2) external field evaluation, (3) calibrated confidence with explicit uncertainty states, (4) visual explanations and (5) a bilingual, source-cited advisory delivered through a deployed web application.

CHAPTER 3

Development Environment and Requirements

3.1 Hardware used for training

Component	Specification
Machine	Apple MacBook with Apple M1 chip (8-core CPU: 4 performance + 4 efficiency cores; 7-core GPU)
Memory	8 GB unified memory (shared by CPU and GPU)
GPU backend	PyTorch MPS (Metal Performance Shaders) — no CUDA GPU available
Free disk space	about 8.5 GB at the start of the project

3.2 Software

Purpose	Software / version
Language	Python 3.12 (ML and backend), TypeScript (frontend), Node.js 22
Deep learning	PyTorch 2.15, torchvision 0.29, timm 1.0.31
Experiment tracking / tuning	MLflow 3.18 (SQLite store), Optuna 5.0
Data / evaluation	NumPy, pandas, Pillow, imagehash, scikit-learn, matplotlib
Serving	ONNX 1.23, ONNX Runtime 1.31, FastAPI, Uvicorn
Web	Next.js 16, React 19, Tailwind CSS 4, shadcn/ui, Recharts
Testing	pytest, Vitest, Playwright 1.64

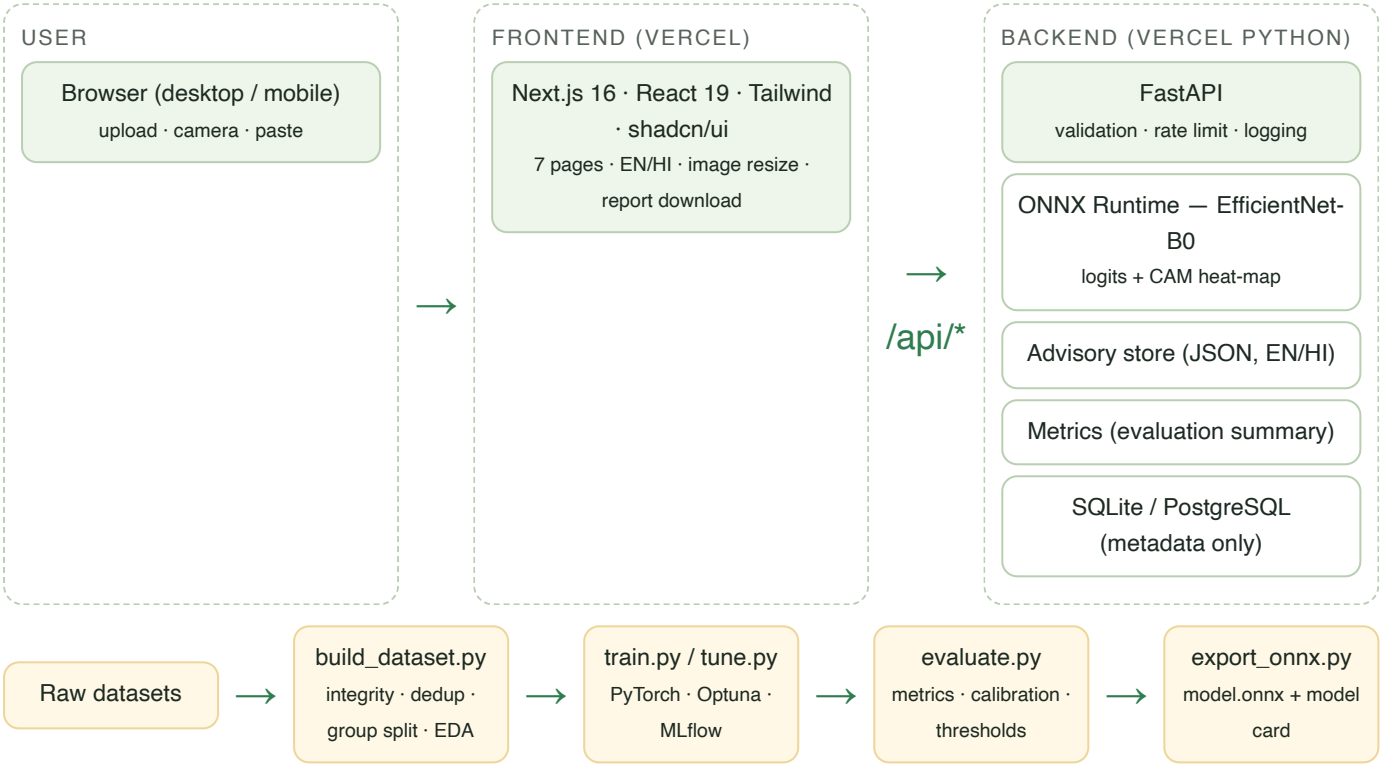
3.3 Measured training throughput (batch 32, 224 px, steady state)

Model	Parameters	Training (images/s)	Inference (images/s)
EfficientNet-B0	4.1 M	34.1	200.7
ResNet-50	23.6 M	22.8	82.6

Mixed-precision (fp16) training on the MPS backend was measured to be about 15 times *slower* than fp32 on this software stack (ResNet-50: 1.6 images/s), so all training used fp32. These measurements guided the epoch budgets used below.

CHAPTER 4

System Architecture and Technology Stack



Architecture diagram. Online request path (top) and offline machine-learning pipeline (bottom).

Layer	Technologies
Deep learning	PyTorch 2, timm (EfficientNet-B0, ImageNet weights), scikit-learn, MLflow, Optuna
Serving	ONNX Runtime (CPU), NumPy, Pillow (+ AVIF), pillow-heif (HEIC)
Backend	FastAPI, Pydantic, SQLAlchemy (PostgreSQL / SQLite), slowapi rate limiting, JSON logging
Frontend	Next.js 16, React 19, TypeScript, Tailwind CSS 4, shadcn/ui, Lucide icons, Recharts
Testing	pytest, Vitest + Testing Library, Playwright (Chrome, desktop + mobile emulation)
Deployment	Vercel (Hobby plan): Next.js project + Python (FastAPI) project; Dockerfiles + docker-compose also provided

CHAPTER 5

Datasets and Exploratory Data Analysis

5.1 Sources

Dataset	Role	Images	Classes	Licence	Source
PlantVillage (colour)	Train / validation / internal test	54,305	38	CC BY-SA 3.0	github.com/spMohanty/PlantVillage-Dataset
PlantDoc	External validation + external test (field)	2,572	28 → mapped	CC BY 4.0	github.com/pratikkayal/PlantDoc-Dataset
Tomato Leaf Disease, Pakistan	Independent external test (field)	7,200	6 → mapped	CC BY 4.0	Mendeley Data 3mbnb82mxd

All datasets were downloaded automatically from the original authors' repositories (Kaggle required an API token) and verified file by file. Every PlantDoc and Tomato-field class was mapped by hand onto the matching PlantVillage class so that external evaluation compares like with like.

5.2 Integrity checks

Check	Result
Corrupt / unreadable images	0 of 64,077
PlantVillage image size	all 256×256 RGB JPEG
Exact duplicates (MD5), all sources	182 groups / 381 images
Images matched to the PlantVillage leaf-identity map	41,111 (7,257 leaves have several photos)
Near-duplicate pairs (perceptual hash, ≤ 4 bits)	1,165
Leakage groups after merging	19,988
PlantDoc ↔ PlantVillage near-duplicates	1 (removed)
PlantDoc test images duplicating PlantDoc train	14 (removed)

5.3 Class balance

The dataset is strongly imbalanced: the largest class (*Orange — Haunglongbing (Citrus greening)*) has 5,507 images and the smallest (*Potato — healthy*) only 152 — a ratio of **36.23×**. This is handled with class-weighted loss and macro-averaged metrics.

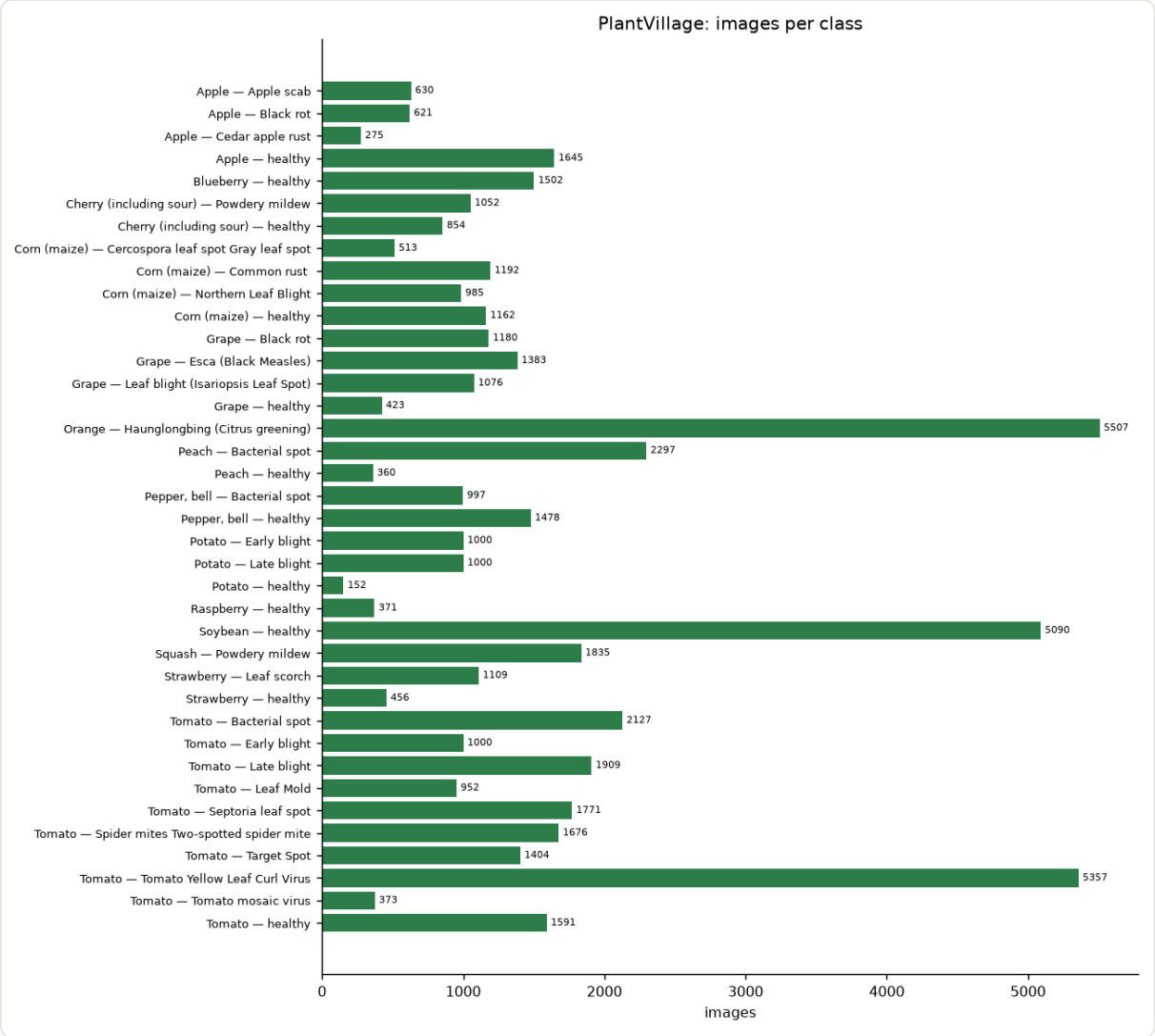


Figure 1. Number of images per PlantVillage class.

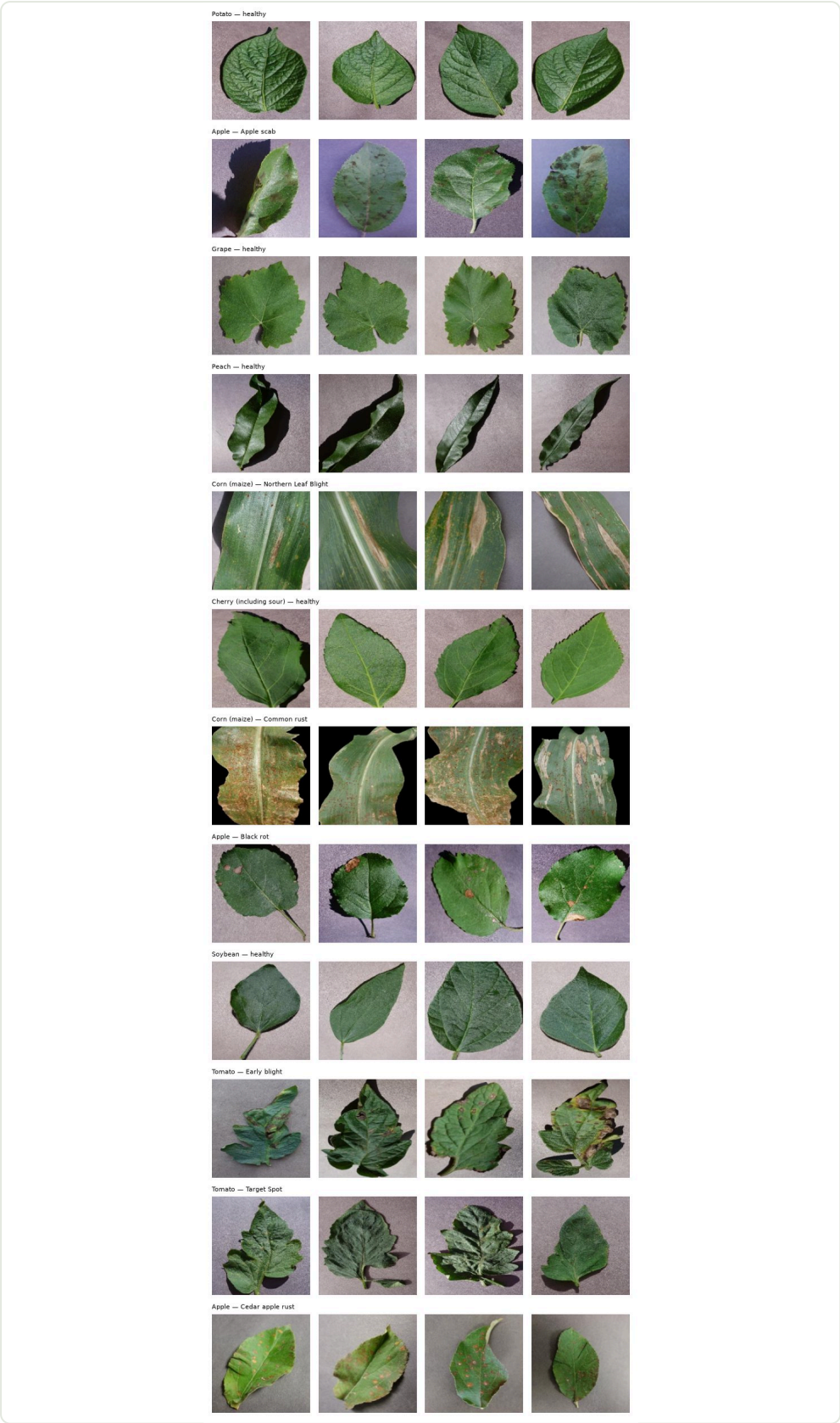


Figure 2. Random PlantVillage samples: single leaves on a uniform background (lab-style).



Figure 3. PlantDoc samples: real backgrounds, multiple leaves, varied light.



Figure 4. Tomato field samples (Pakistan), smartphone photos.

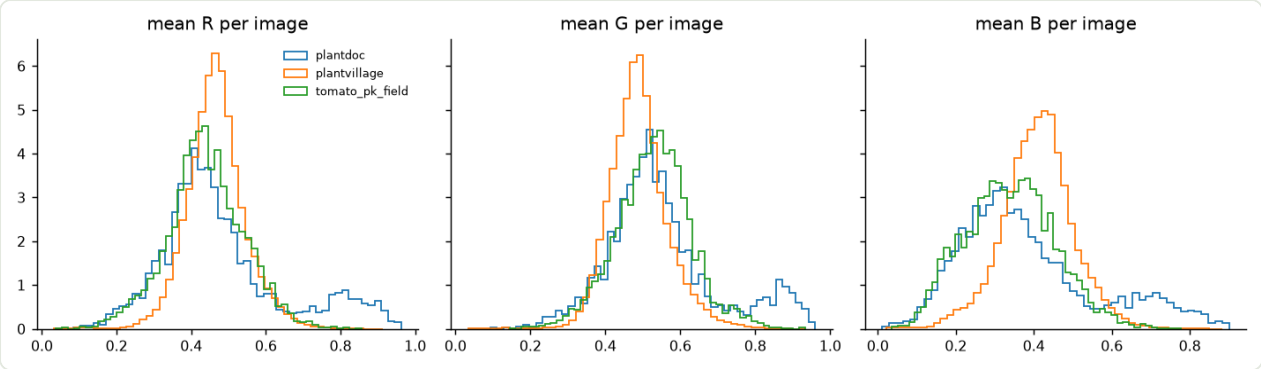


Figure 5. Per-image mean colour by dataset — the field datasets differ clearly from PlantVillage (domain shift).

CHAPTER 6

Data Preprocessing and Leakage-free Splitting

PlantVillage contains several photos of the same physical leaf. A random image-level split would put almost identical photos in both training and test sets and inflate accuracy. Images were therefore grouped by (1) the dataset's own leaf-identity map, (2) exact byte duplicates and (3) perceptual-hash near-duplicates, and **whole groups** were assigned to train / validation / test (≈ 70/15/15, stratified by class, seed 42). A script assertion guarantees that no group crosses splits.

Split	Images	Used for
train	39,309	gradient updates
validation	7,245	early stopping, tuning, calibration, thresholds
test	7,751	final internal evaluation (used once)
external validation (PlantDoc)	463	calibration and thresholds only
domain-train (PlantDoc)	1,872	prepared for future field adaptation (not used)
external test (PlantDoc + Tomato field)	7,422	final external evaluation (used once)

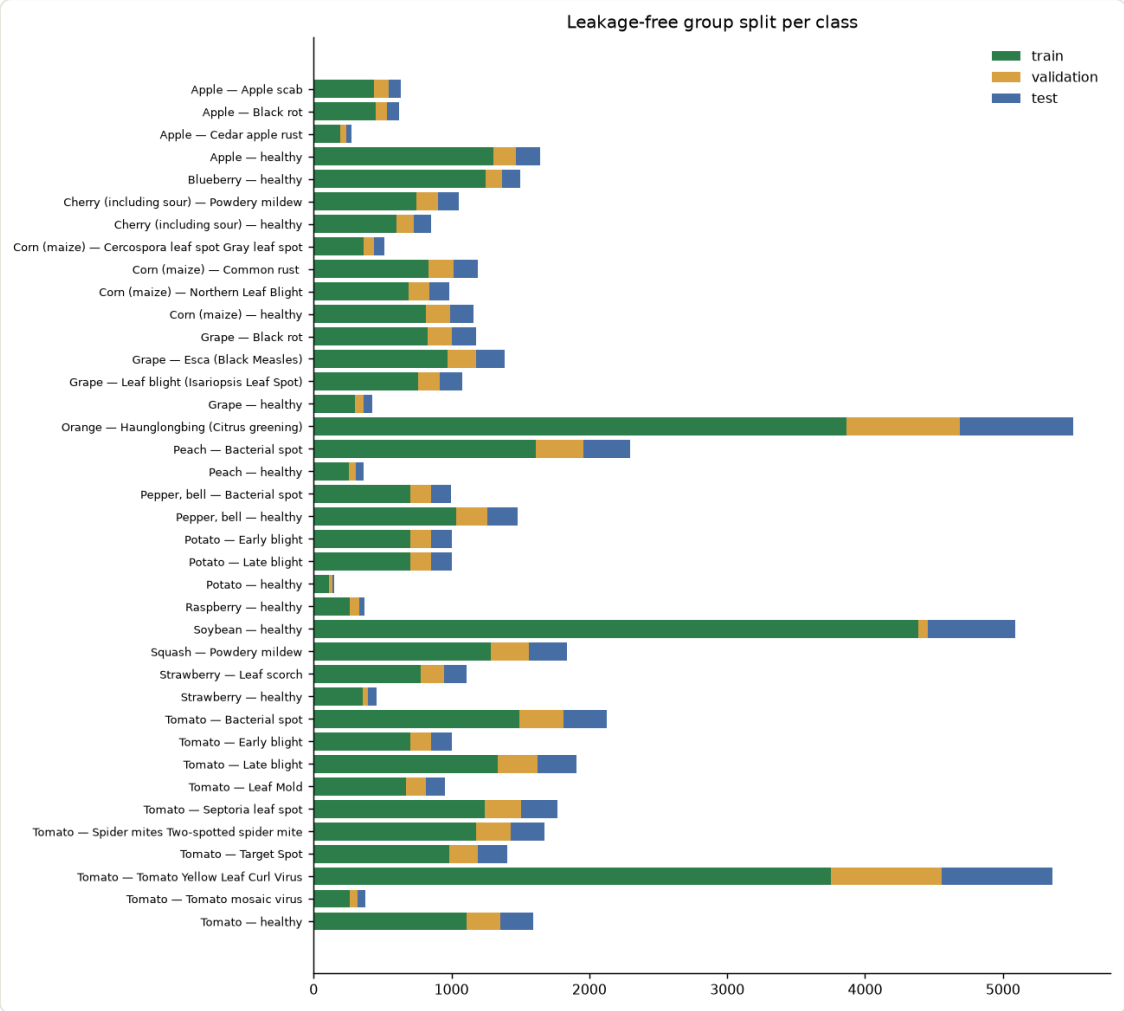


Figure 6. Per-class distribution of the leakage-free split.

6.1 Augmentation and normalisation

- **Training:** random resized crop to 224 px (scale 0.55–1.0), horizontal and vertical flips, rotation up to 30°, brightness / contrast / saturation jitter, very small hue jitter (leaf colour is diagnostic), occasional Gaussian blur.
- **Evaluation and serving:** resize the short side to 256 px, centre-crop 224 px, ImageNet mean/std normalisation. The serving code is unit-tested to produce numerically identical input to the training pipeline.

CHAPTER 7

Model Architecture and Training

7.1 Architecture

EfficientNet-B0 (Tan & Le, 2019) is a compact convolutional network built from mobile inverted-bottleneck (MBConv) blocks with squeeze-and-excitation, scaled for depth/width/resolution. The ImageNet-pretrained backbone (timm) was kept and its classifier replaced by a new 38-class linear layer with dropout 0.2. Parameters: **4.06 M**; input 224×224×3; exported model size 16.5 MB.

7.2 Training setup

Setting	Value
Framework / hardware	PyTorch 2 on Apple M1 GPU (MPS, fp32), 8 GB unified memory
Optimiser	AdamW; head LR 1.95e-03, backbone LR ×0.40, weight decay 0.0181
Schedule	cosine decay with 5 % linear warm-up, per batch
Loss	class-weighted (inverse √frequency) cross-entropy, label smoothing 0.12
Batch size / epochs	32 / 5 (early stopping on validation macro-F1, patience 2)
Training time	103 minutes (5 epochs)
Best epoch	5
Tracking	MLflow (sqlite) — every epoch logged

7.3 Training procedure (step by step)

1. **Load data:** each batch of 32 images is read from the split manifest, decoded, augmented (random resized crop, flips, rotation, colour jitter, blur) and normalised; 3 parallel data-loader workers feed the GPU.
2. **Model:** EfficientNet-B0 with ImageNet weights; the 1000-class head is replaced by a 38-class linear layer.
3. **Fine-tuning:** all layers are trained. The new head uses the base learning rate; the pretrained backbone uses a smaller rate (×0.40) so that ImageNet features are adapted gently.
4. **Loss:** cross-entropy with label smoothing (0.12) and per-class weights (inverse square root of class frequency) to counter the 36× class imbalance.
5. **Schedule:** learning rate rises linearly for the first 5 % of steps, then follows a cosine decay to zero.
6. **Validation every epoch:** accuracy, macro-F1 and weighted-F1 on the PlantVillage validation split and on the PlantDoc external-validation split are logged to MLflow; the checkpoint with the best validation macro-F1 is kept.
7. **Early stopping:** training stops if validation macro-F1 does not improve for 2 epochs (here all 5 epochs ran because the score kept improving).

7.4 Engineering issues during training and how they were solved

Problem observed	Cause found	Solution
Training extremely slow with mixed precision	fp16 autocast on the MPS backend ~15× slower	train in fp32; mixed precision enabled only on CUDA
Training hung on the first batch	weighted, label-smoothed cross-entropy on MPS forces a GPU sync and stalled under memory pressure	equivalent loss written with gather/sum (verified identical to PyTorch to 1e-6)
Very low throughput (< 1 image/s) in a smoke test	too many persistent data-loader worker processes (~325 MB each) on 8 GB RAM	3 training workers; validation workers created only during evaluation

Problem observed	Cause found	Solution
Experiment tracking failed	MLflow 3 refuses the file-based store	MLflow with a SQLite backend

7.5 Summary of all experiments performed

#	Experiment	Data used	Configuration	Time	Outcome
E1	Hardware benchmark	random tensors	EfficientNet-B0 and ResNet-50, fp32 vs fp16, batch 32	minutes	fp32 chosen; B0 34 img/s, ResNet-50 23 img/s
E2	Hyper-parameter search (Optuna, TPE)	4,000 train / 1,500 val images (stratified)	8 trials × 1 epoch; LR, backbone multiplier, weight decay, label smoothing	28 min (training only)	best val macro-F1 0.889 (trial 4)
E3	ResNet-50 learning-rate probe (head LR 0.001)	6,000 train / 1,500 val images	1 epoch, backbone LR ×0.1	6.2 min	val macro-F1 0.400
E4	ResNet-50 learning-rate probe (head LR 0.003)	6,000 train / 1,500 val images	1 epoch, backbone LR ×0.1	6.7 min	val macro-F1 0.863
E5	**EfficientNet-B0 final training**	39,309 train images per epoch	Optuna-best settings, 5 epochs, batch 32	103 min	**val acc 99.72%, val macro-F1 0.9963**
E6	Evaluation pass (inference only)	22,881 images: validation, external validation, test, external test	temperature fit, thresholds, metrics, plots	≈ 4 min	test acc 99.56%
E7	ONNX export and verification	105 test images	PyTorch → ONNX (opset 17), CAM output	< 1 min	identical predictions; CAM ≡ Grad-CAM

Planned but not run. Full ResNet-50 training (3 epochs), a Vision Transformer (ViT-S/16, 3 epochs), EfficientNet-B2 (2 epochs) and an EfficientNet-B0 domain-adaptation run on PlantDoc images were prepared in the training queue but stopped to keep the computational load suitable for a laptop. The trained EfficientNet-B0 already met the project objectives.

7.6 Hyper-parameter search (Optuna)

8 TPE trials, each fine-tuning for one epoch on a stratified 4,000-image subset and scored by macro-F1 on 1,500 validation images. The best trial (bold) was used for the full run. Trials with a very low backbone learning rate barely learned in one epoch.

Trial	Val macro-F1	Head LR	Backbone ×	Weight decay	Label smoothing
0	0.8272	8.61e-04	0.435	5.1e-03	0.090
1	0.1727	4.65e-04	0.047	1.6e-05	0.130
2	0.8167	1.63e-03	0.220	1.2e-05	0.145
3	0.7854	3.12e-03	0.055	4.7e-05	0.028
4	**0.8895**	**1.95e-03**	**0.401**	**1.8e-02**	**0.120**
5	0.1658	4.06e-04	0.060	8.1e-03	0.138
6	0.8670	4.46e-03	0.236	3.8e-03	0.088
7	0.2419	4.18e-04	0.091	3.8e-02	0.023

A short ResNet-50 learning-rate probe (1 epoch, 6,000 images) was also run: head LR 1e-3 → validation macro-F1 0.400; head LR 3e-3 → 0.863. Full ResNet-50 and Vision-Transformer training was not carried out to keep the computational load suitable for a laptop.

7.7 Training history

Epoch	Train loss	Train acc (aug.)	Val loss	Val acc	Val macro-F1	Field ext-val macro-F1	Minutes
1	1.464	88.95%	0.218	98.80%	0.9858	0.1971	19.3
2	1.123	97.94%	0.262	98.48%	0.9797	0.1746	23.7
3	1.074	98.92%	0.212	99.19%	0.9887	0.2164	19.4
4	1.045	99.45%	0.189	99.68%	0.9958	0.1805	22.6
5	1.031	99.74%	0.192	99.72%	0.9963	0.1676	18.4

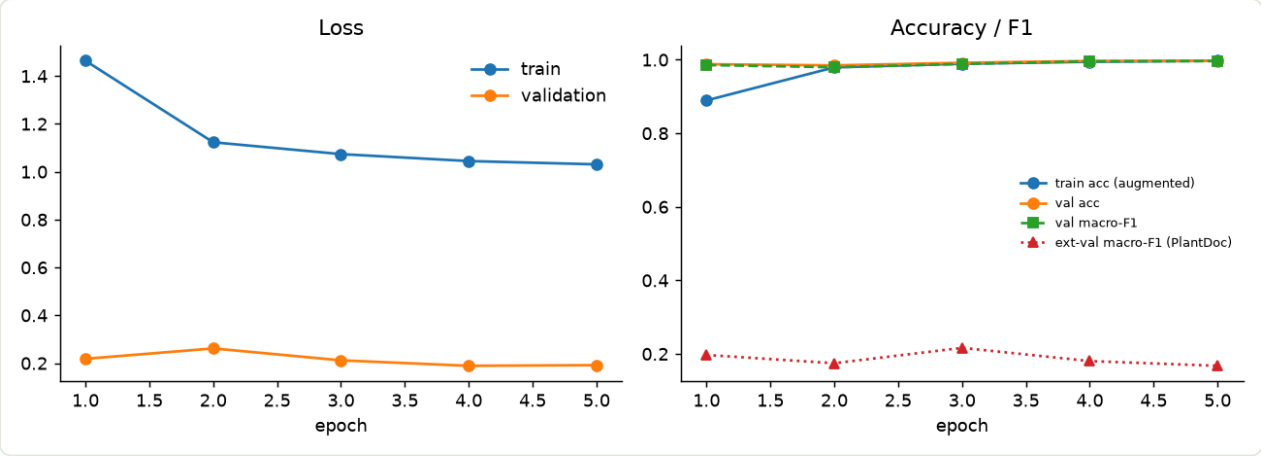


Figure 7. Training curves: loss (left) and accuracy / macro-F1 (right). Validation accuracy is near its ceiling after epoch 1; field (PlantDoc) performance stays low throughout.

CHAPTER 8

Confidence Calibration and Uncertainty Handling

A softmax score is not a guarantee of correctness. The system therefore:

1. fits a single **temperature** T = 1.055 on the validation sets (lab and field weighted equally);
2. selects a **confident threshold** $\tau = 0.730$ and a **reject threshold** $\tau_r = 0.647$ on validation data only (rule: target unreachable on some domain -> threshold maximising balanced selective accuracy at >=10% coverage);
3. reports *confident* ($p \geq \tau$), *uncertain* ($\tau_r \leq p < \tau$, advice hidden, top-3 shown) or *not recognised* ($p < \tau_r$);
4. never reports a blurry photo (variance-of-Laplacian below the 1st validation percentile, 36.99) as confident.

Set	Answered (coverage)	Accuracy on answered	Not recognised	ECE raw → calibrated
PlantVillage validation	77.5%	99.9%	1.5%	0.166 → 0.205
PlantDoc ext-validation	12.3%	43.9%	78.4%	0.265 → 0.235
PlantVillage test	72.1%	99.9%	1.5%	0.172 → 0.211
PlantDoc test	16.2%	50.0%	71.6%	0.196 → 0.171
Tomato field test	18.0%	47.5%	69.9%	0.258 → 0.225

On lab-style photos 72 % of images are answered confidently at 99.9 % accuracy. On field photos most images (~70 %) are honestly reported as *not recognised*, but confident field answers are still only about 50 % correct — a limitation that the application states explicitly.

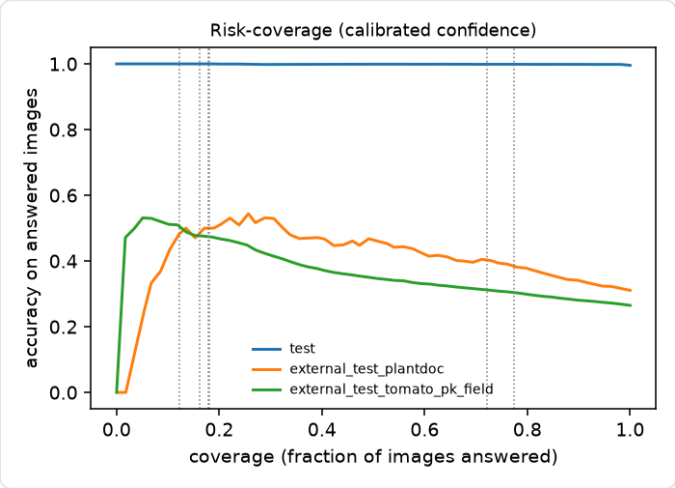


Figure 8. Risk–coverage: accuracy on the images the model answers as the confidence bar is raised.

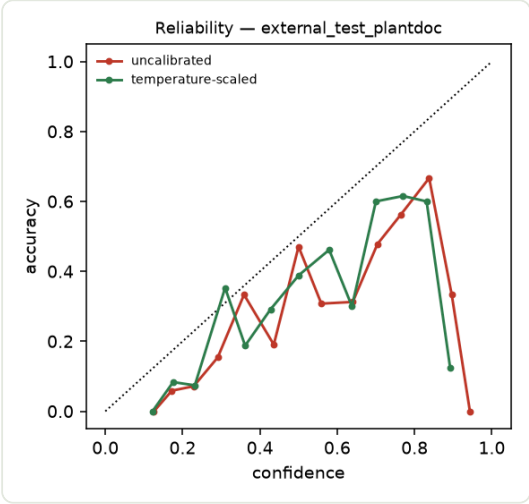


Figure 9. Reliability diagram (PlantDoc test) before and after temperature scaling.

CHAPTER 9

Results and Evaluation

9.1 Internal test set (PlantVillage, never used for training or selection)

99.56%	99.46%	99.52%	99.49%	99.56%
Accuracy	Macro precision	Macro recall	Macro F1	Weighted F1

Only **34 of 7,751** test images are misclassified. Crop identification accuracy is 99.96% and top-3 accuracy 99.97%.

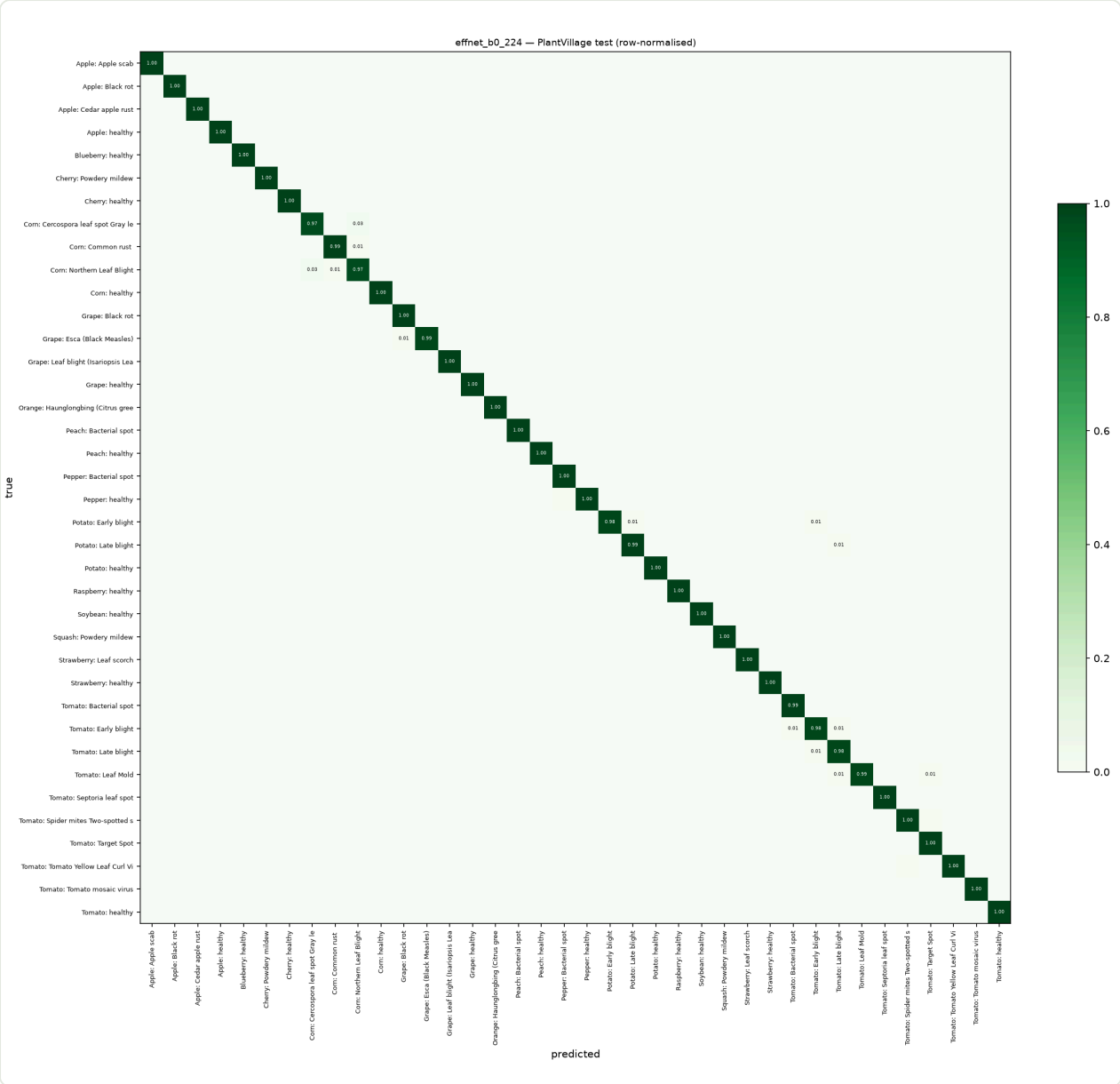


Figure 10. Row-normalised confusion matrix on the PlantVillage test set (38 classes).

9.2 All evaluation sets

Set	n	Accuracy	Macro P	Macro R	Macro F1	Weighted F1	Crop acc.	Top-3
PlantVillage validation	7,245	99.72%	99.58%	99.70%	99.63%	99.72%	100.0%	100.0%
PlantVillage test	7,751	99.56%	99.46%	99.52%	99.49%	99.56%	100.0%	100.0%

Set	n	Accuracy	Macro P	Macro R	Macro F1	Weighted F1	Crop acc.	Top-3
PlantDoc test (field)	222	31.08%	35.25%	31.41%	28.33%	28.84%	58.1%	53.6%
Tomato field test (Pakistan)	7,200	26.50%	48.59%	26.50%	24.22%	24.22%	70.8%	44.0%

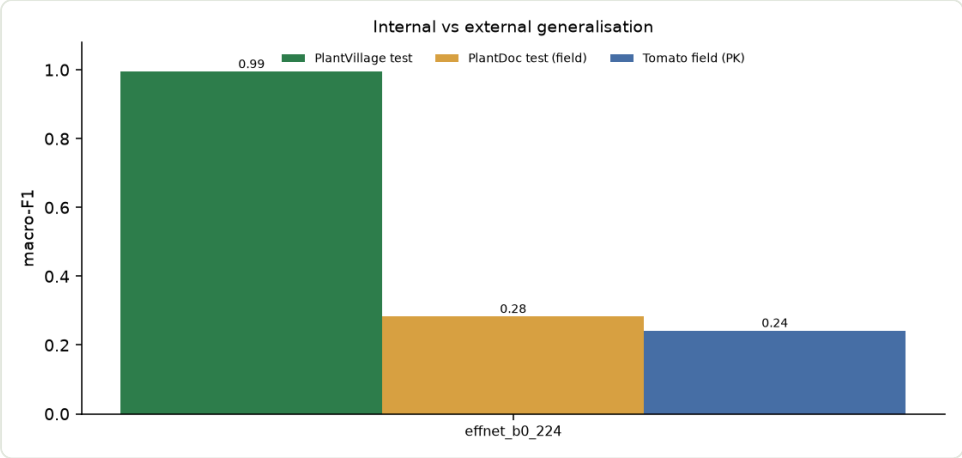


Figure 11. Internal (lab) vs external (field) macro-F1 of the trained model.

9.3 External (field) evaluation

The drop from lab to field photos (99.6% → 31.1% on PlantDoc and 26.5% on the tomato field set) is the classic dataset-bias result reported by the PlantDoc authors: a model trained only on single leaves photographed on a plain background does not transfer to cluttered field photos.

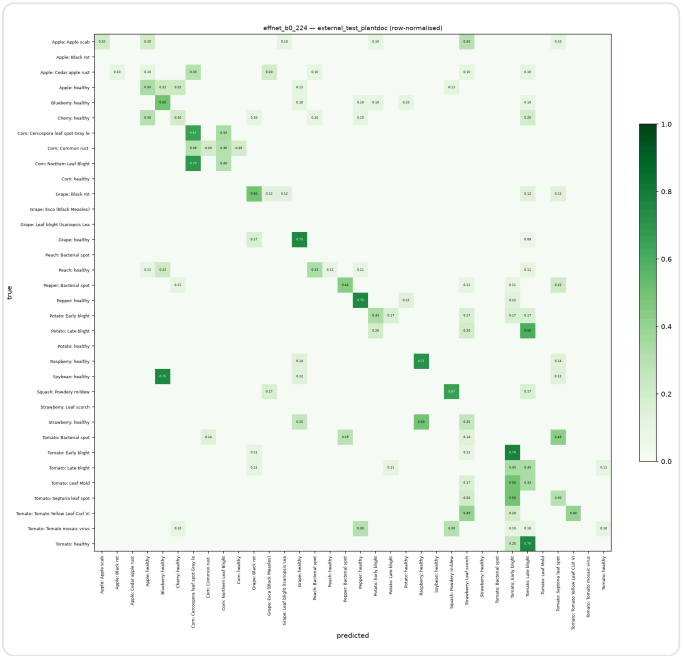


Figure 12. Confusion matrix — PlantDoc test (classes present).

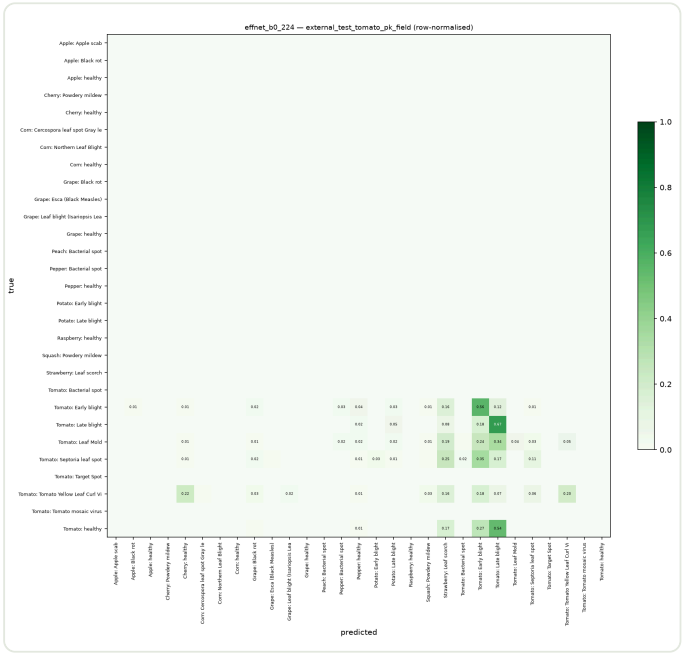


Figure 13. Confusion matrix — Tomato field test.

9.4 Error analysis

- **Internal:** errors occur between visually similar lesions on the same crop — e.g. Tomato — Tomato Yellow Leaf Curl Virus → Spider mites Two-spotted spider mite (4); Tomato — Late blight → Early blight (4); Corn — Northern Leaf Blight → Cercospora leaf spot Gray leaf spot (4); Grape — Esca (Black Measles) → Black rot (2).

- **PlantDoc:** healthy leaves on the plant are confused with other crops' healthy classes or with blights; crop-level accuracy is only 58.1%.
- **Tomato field:** healthy field leaves are called Late or Early blight; Septoria ↔ Early blight and Leaf Mold ↔ Late blight are the main confusions — natural light, shadows and soil look like lesions to a lab-trained model.

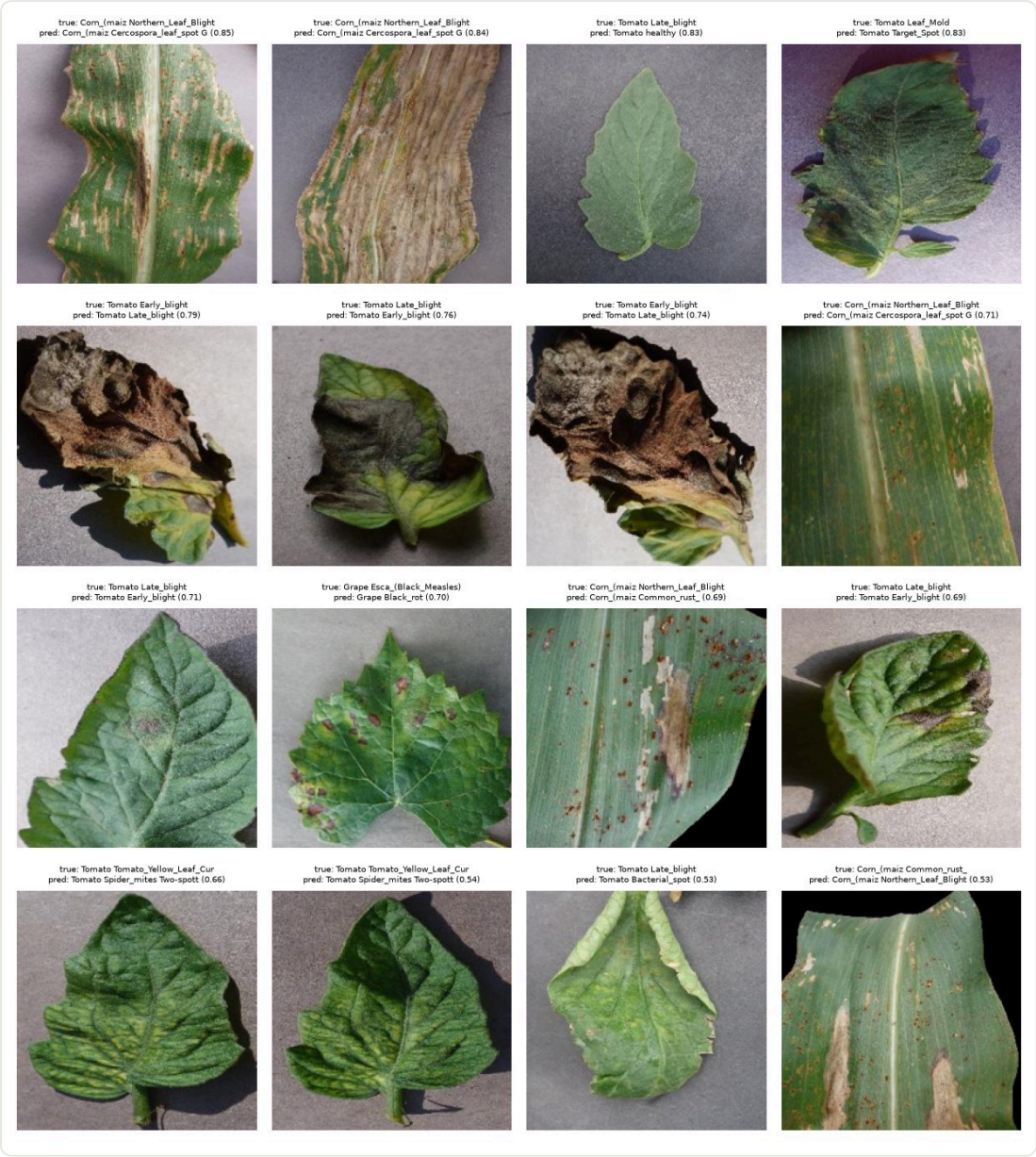


Figure 14. The most confident mistakes on the PlantVillage test set (true vs predicted, calibrated confidence).

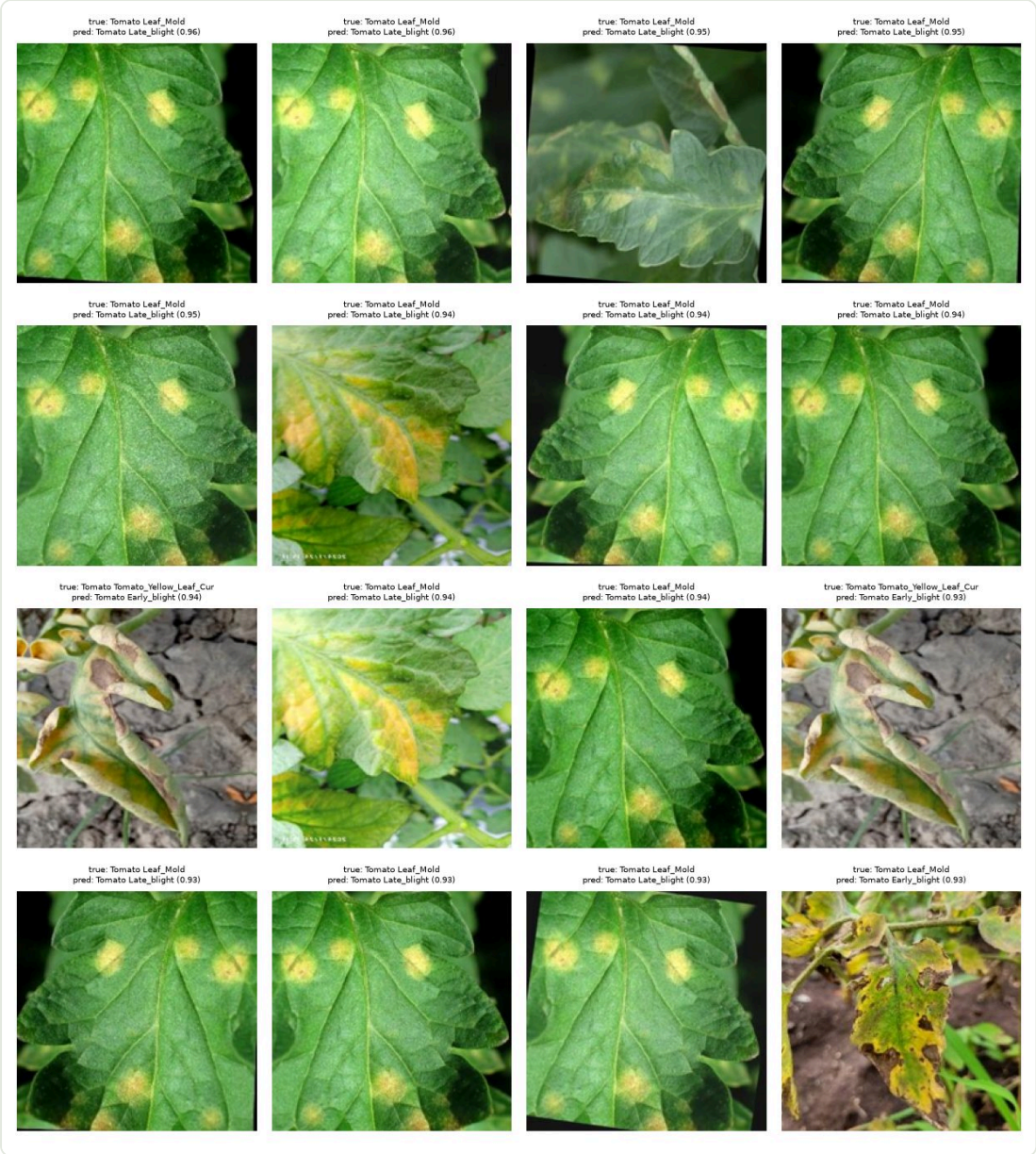


Figure 15. The most confident mistakes on tomato field photos.

CHAPTER 10

Explainability (Grad-CAM / CAM)

Grad-CAM (Selvaraju et al., 2017) highlights the image regions that most influenced the predicted class score. EfficientNet-B0 ends with global average pooling and one linear layer, so Grad-CAM at the last feature map equals the classic **Class Activation Map** (CAM = classifier weights × feature maps) up to a constant factor. The deployed ONNX model therefore outputs the CAM directly, without gradients. This equivalence was verified numerically: minimum correlation between CAM and Grad-CAM = **1.00000000**; ONNX predictions agree with PyTorch on **105** test images (max logit difference 1.0e-05).

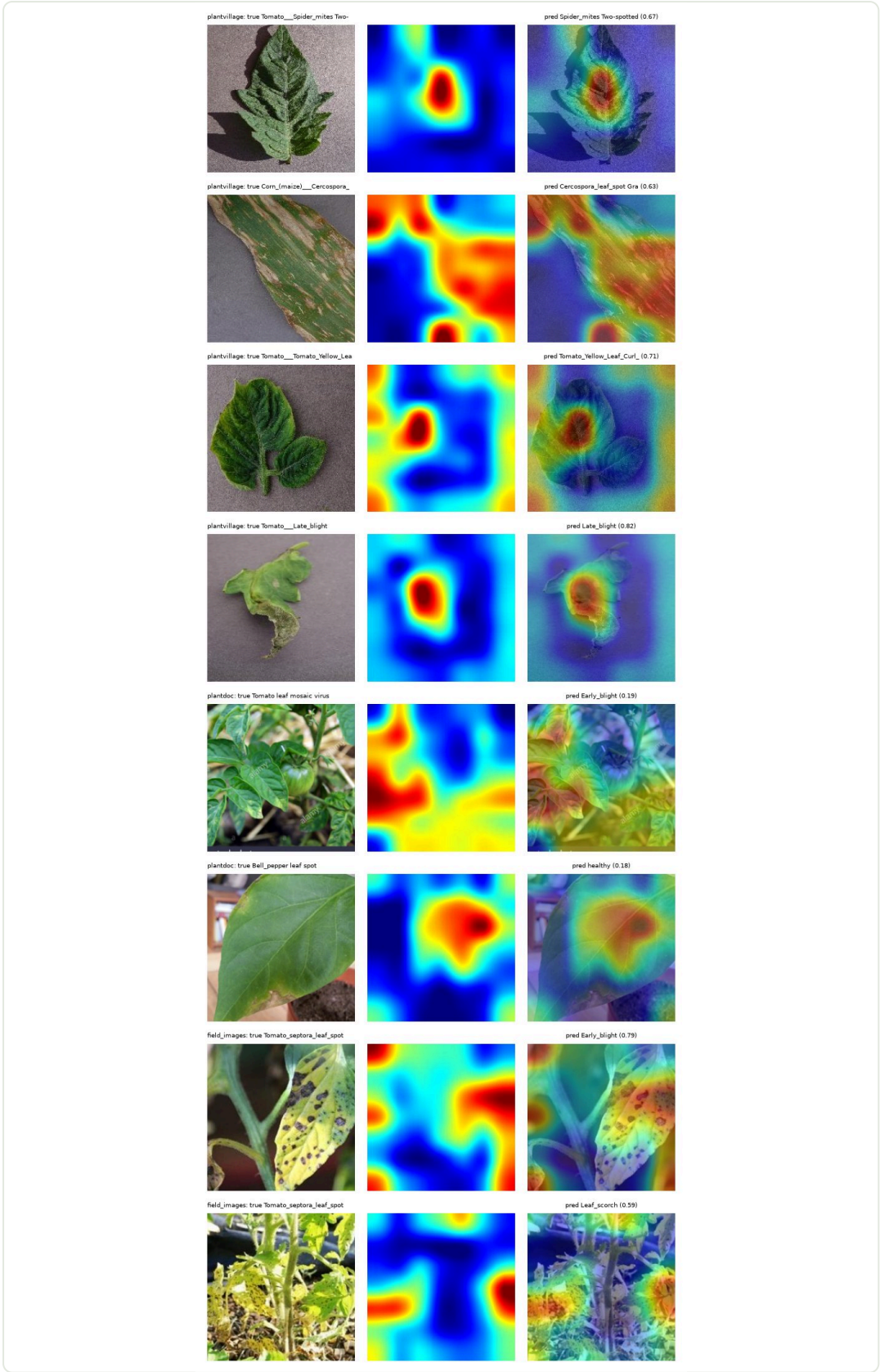


Figure 16. Model input, heat-map and overlay for lab (top four rows) and field (bottom four rows) images. On lab images the model focuses on the leaf and lesions; on field photos attention is scattered and predictions are wrong with low confidence.

Heat-maps show influence on the class score; they are a sanity check, not proof that a diagnosis is correct.

CHAPTER 11

Bilingual Advisory Engine

A structured JSON knowledge base (`advisory/diseases.json`) holds one entry for each of the 38 classes with crop and disease names, scientific name, pathogen type, summary, symptoms, causes, prevention, organic options and chemical options — every field in **English and simple Hindi**. It cites **52 distinct sources** (ICAR / NIPHM integrated pest management packages, TNAU Agritech, Kerala Agricultural University, National Horticulture Board, FAO and university extension services), each fetched and dated.

- **No dosages** are ever given; chemical options are listed only as "options reported by <source>" and every list ends with an instruction to use products registered by CIB&RC, follow the label and consult the local KVK.
- Advice is shown **only for confident predictions**; for uncertain results the user sees candidate diseases with links.
- A safety review replaced an older recommendation of streptocycline (an antibiotic the Government of India has moved to prohibit in agriculture) with a "do not use" warning.

Example entry — Tomato, Early blight (*Alternaria solani*):

Field	Content (English version; the Hindi version is stored in parallel)
Summary	Early blight is a common fungal disease of tomato that causes brown spots with concentric 'bull's-eye' rings, usually starting on older leaves. It can defoliate plants and also rots fruit near the stem end in warm, humid weather.
Symptoms	• Small dark spots on older leaves that enlarge with concentric rings (bull's-eye pattern). • Leaf tissue around the spots turns yellow; much of the foliage may die in hot, humid weather. • Similar lesions on stems can girdle the plant near the soil. • Fruit develops dark ringed spots, usually near the calyx or stem attachment.
Causes	• Caused by the fungus Alternaria solani. • Survives in infected plant debris and can be seed-borne. • Spores spread by rain, wind, insects, people and machinery. • Warm, humid, rainy weather favours the disease.
Prevention	• Remove and destroy crop debris and practise crop rotation. • Use resistant or tolerant varieties and healthy seed. • Change the nursery location every season and remove weeds and volunteer tomato plants. • Fertilize properly and avoid planting overlapping tomato crops in adjacent fields.
Organic options	• Seed treatment with Trichoderma asperellum is an option reported by TNAU; it may help reduce seed-borne infection. • Removing infected lower leaves and keeping the field clean may help slow spread.
Chemical options	• Fungicide options reported by TNAU and NIPHM include mancozeb, zineb, metiram, copper oxychloride, captan, iprodione, azoxystrobin and pyraclostrobin. • Combination products reported by TNAU include azoxystrobin + difenoconazole, metiram + pyraclostrobin and tebuconazole + trifloxystrobin. • Alternate fungicide groups to slow resistance. • Use only products registered for this crop in India (CIB&RC), follow the product label exactly, and consult your local KVK or agriculture officer before spraying.
Sources	TNAU Agritech Portal, Tamil Nadu Agricultural University NIPHM / Directorate of Plant Protection, Quarantine & Storage, Govt. of India

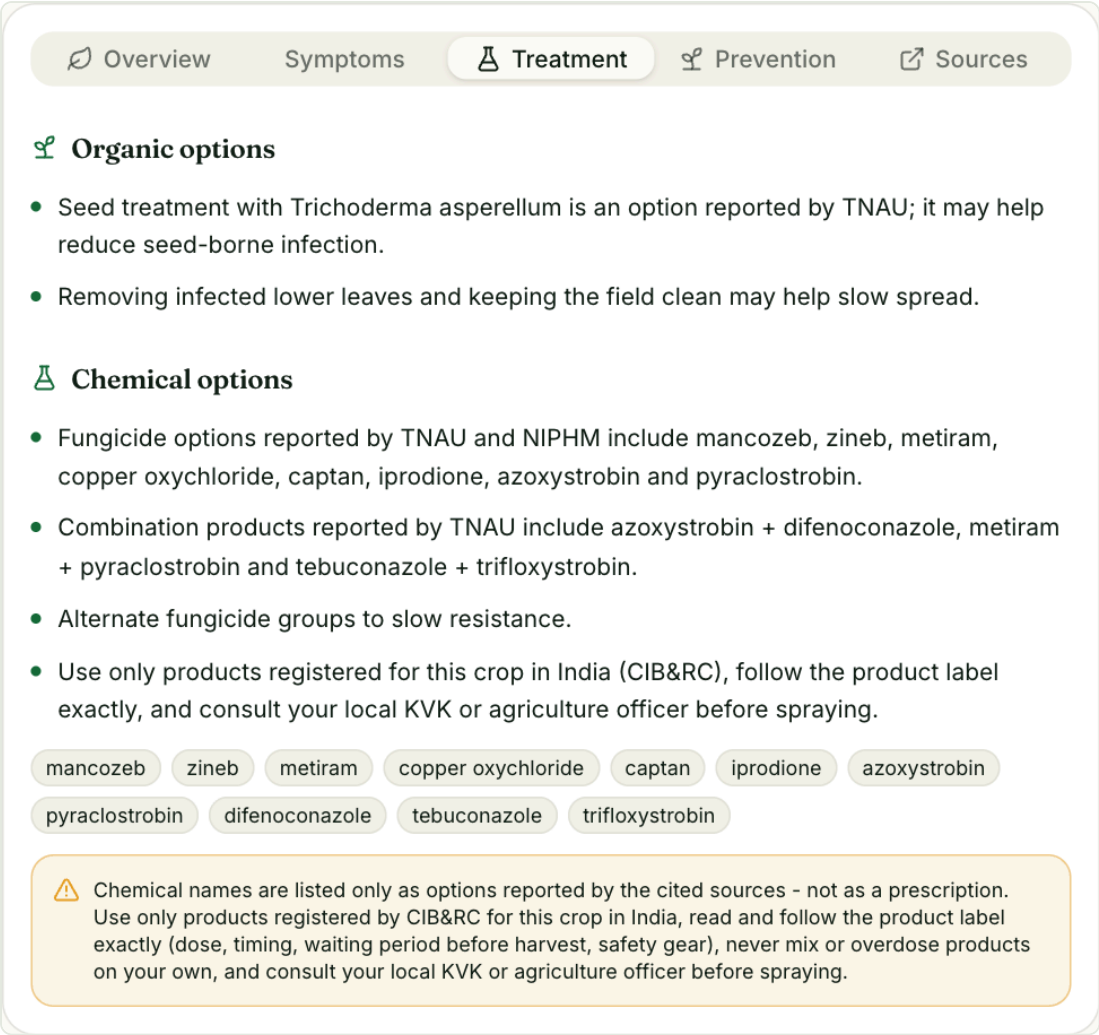


Figure 17. Treatment tab in the application with organic and chemical options and the label-safety notice.

CHAPTER 12

Web Application

The website has seven pages: Landing, Diagnose, Results, Disease library, Model performance, Methodology and About. It is fully responsive and bilingual, supports drag-and-drop, file selection, browser camera capture and **pasting an image (Ctrl/⌘+V or a Paste button)**, and accepts JPEG, PNG, WebP, AVIF, HEIC (iPhone), BMP, TIFF and GIF. Large photos are resized in the browser before upload.

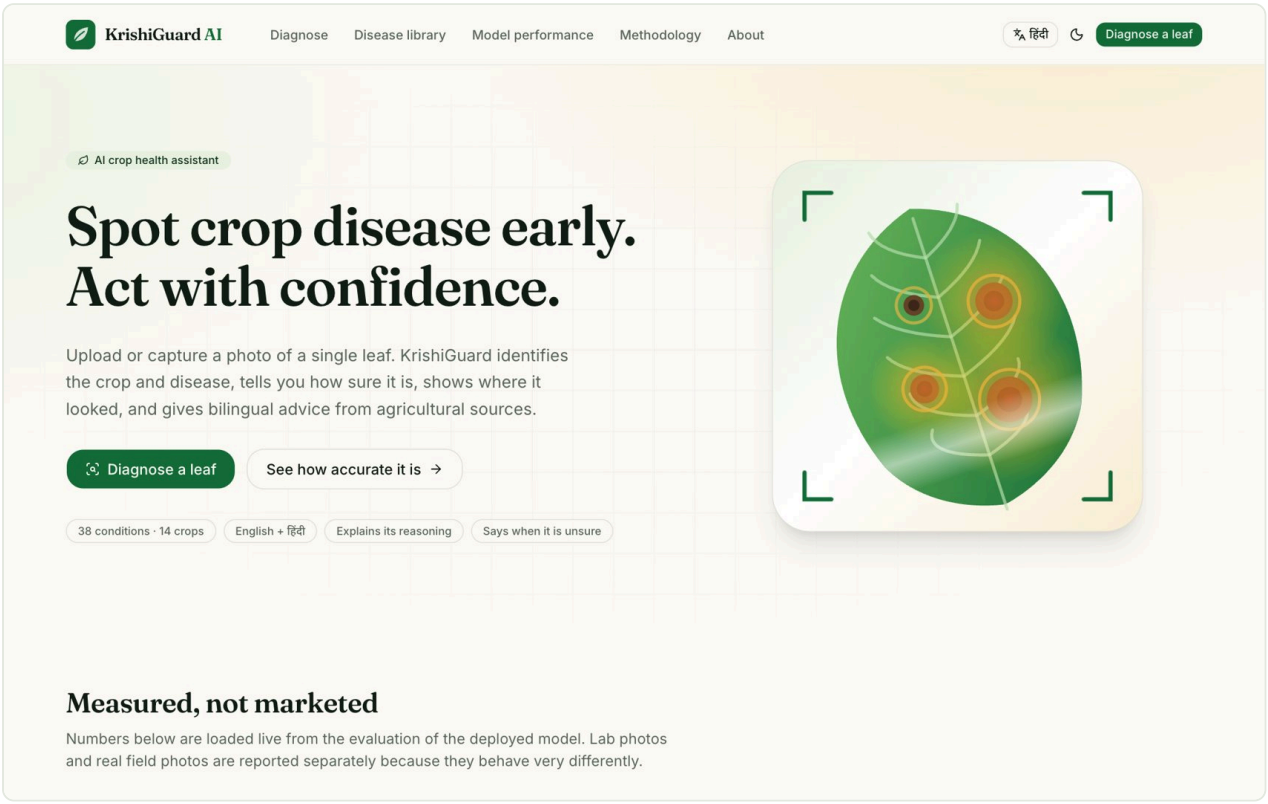


Figure 18. Landing page — live metrics are loaded from the evaluation results.

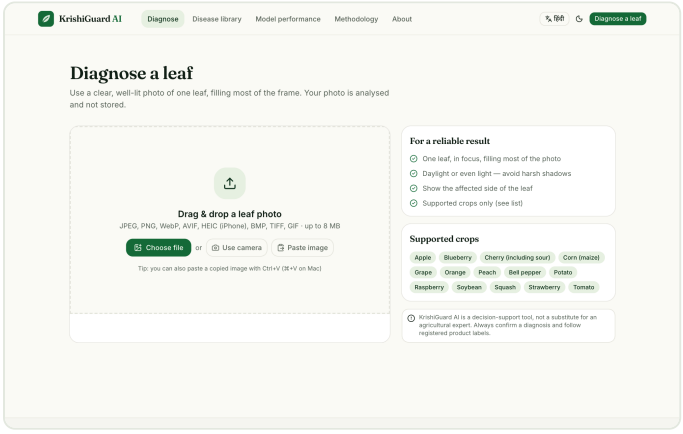


Figure 19. Diagnose page: upload, camera and paste options, supported formats.

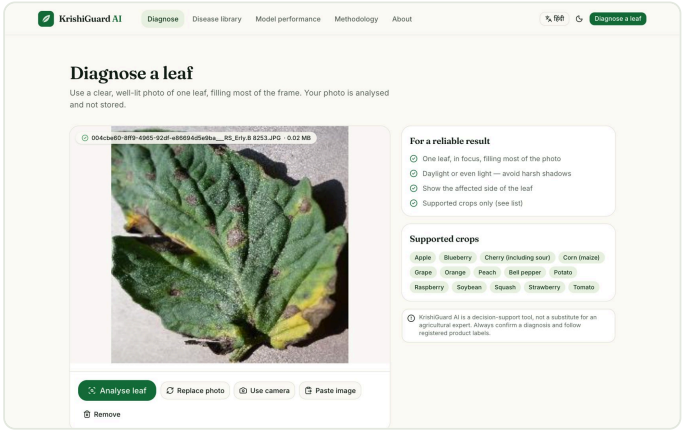


Figure 20. Selected photo with preview, replace / camera / paste / remove actions.

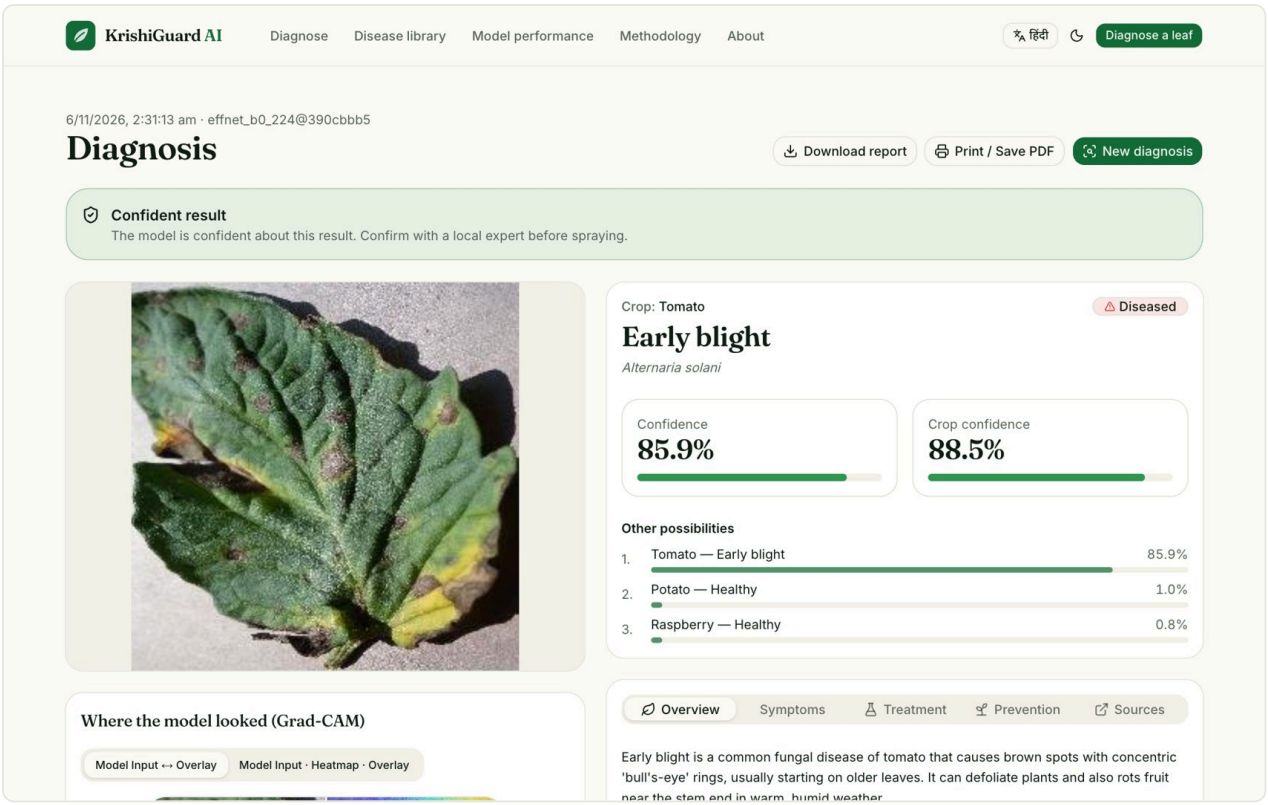


Figure 21. Results: status banner, crop and disease, calibrated confidence, top-3 candidates, heat-map and advisory.

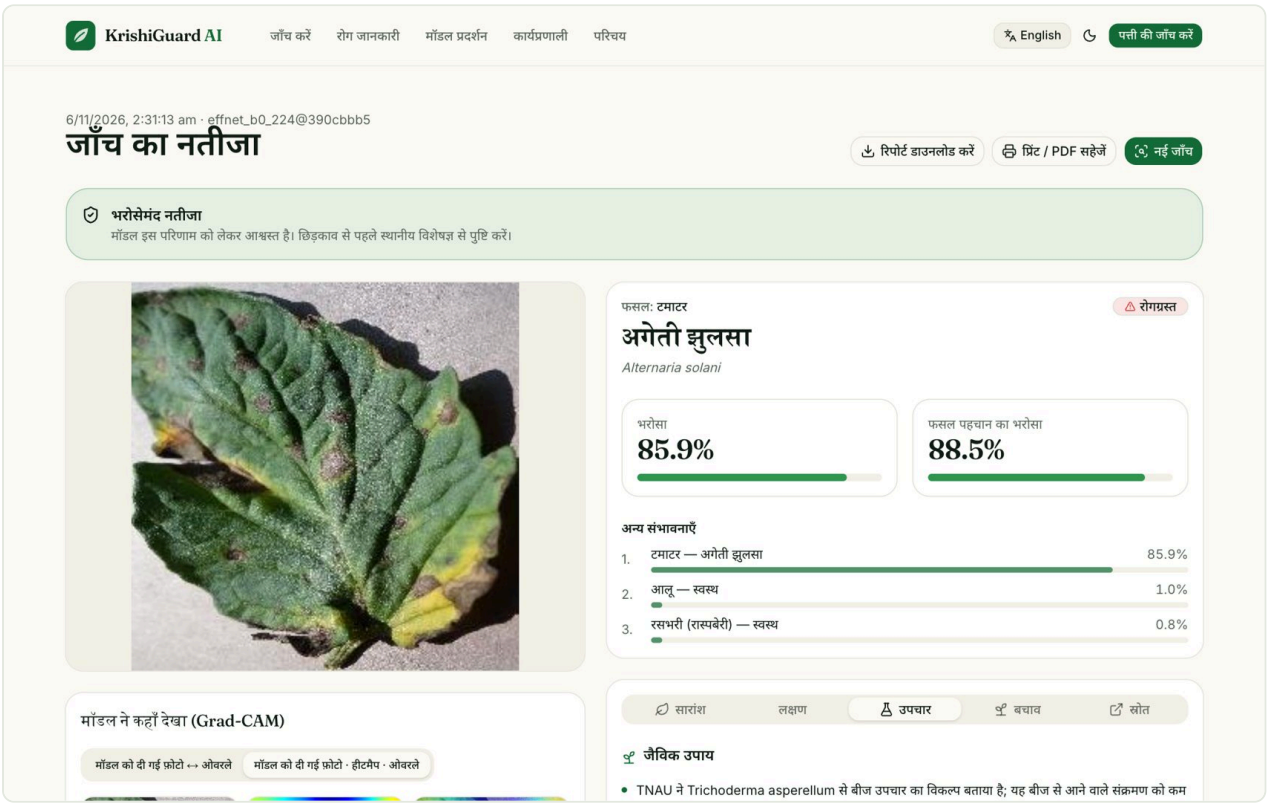


Figure 22. The same result after switching to Hindi.

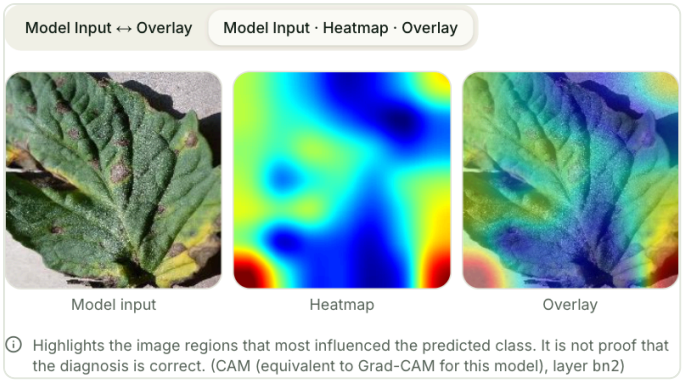


Figure 23. Heat-map viewer: model input, heat-map and overlay (also a slider compare mode).

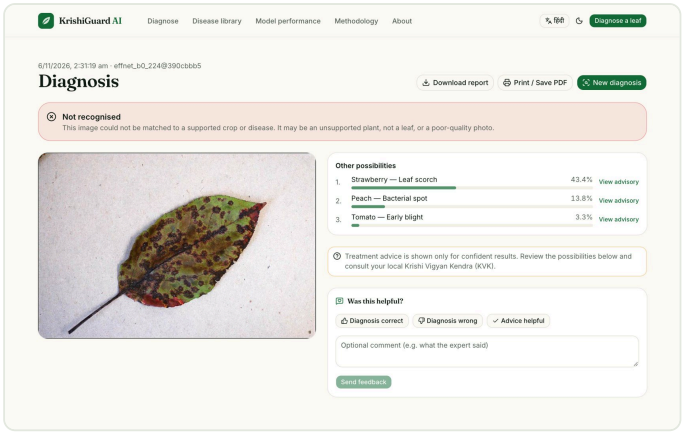


Figure 24. A field photo outside the training distribution is reported as not recognised; no advice is shown.

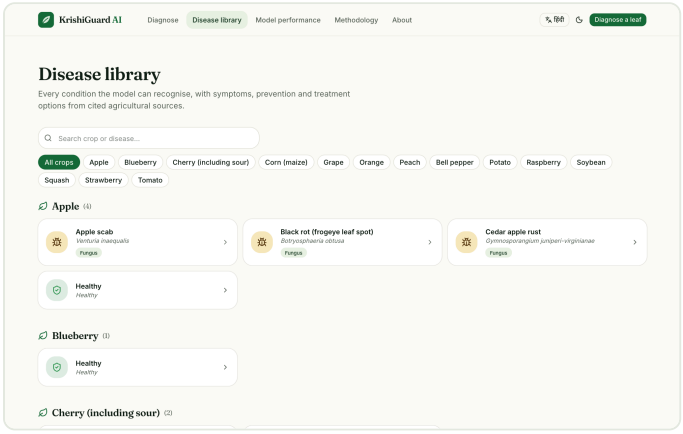


Figure 25. Disease library with search and crop filters.

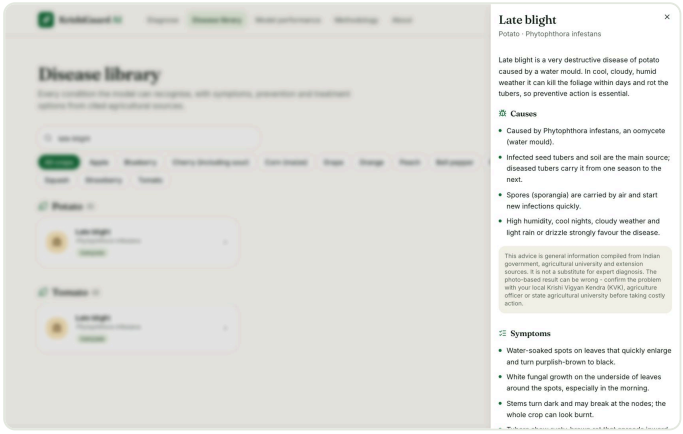


Figure 26. Disease detail sheet with cited advisory.

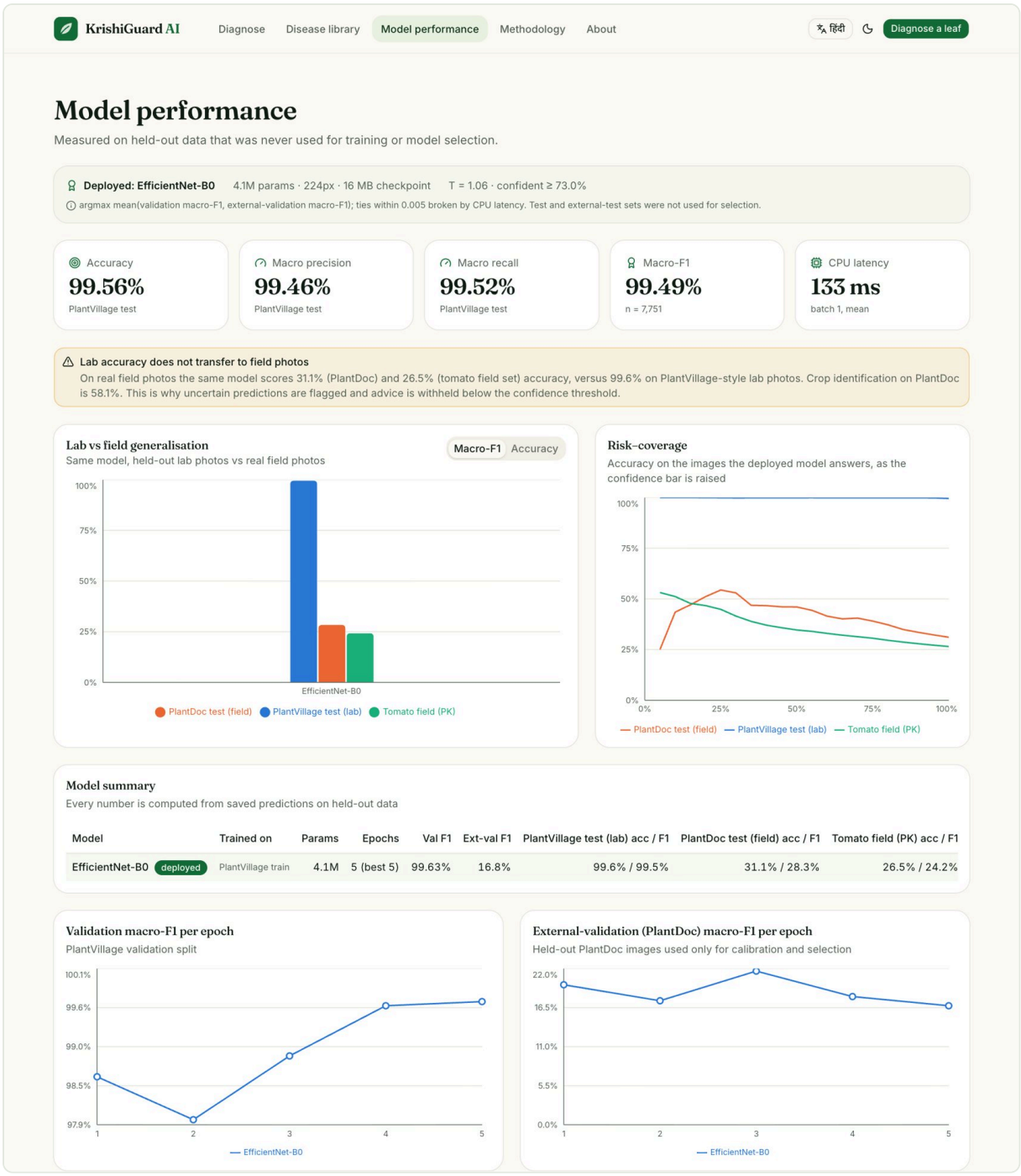


Figure 27. Model performance dashboard (top part) — KPIs, lab-vs-field chart, risk–coverage and model summary; every number comes from the real evaluation.

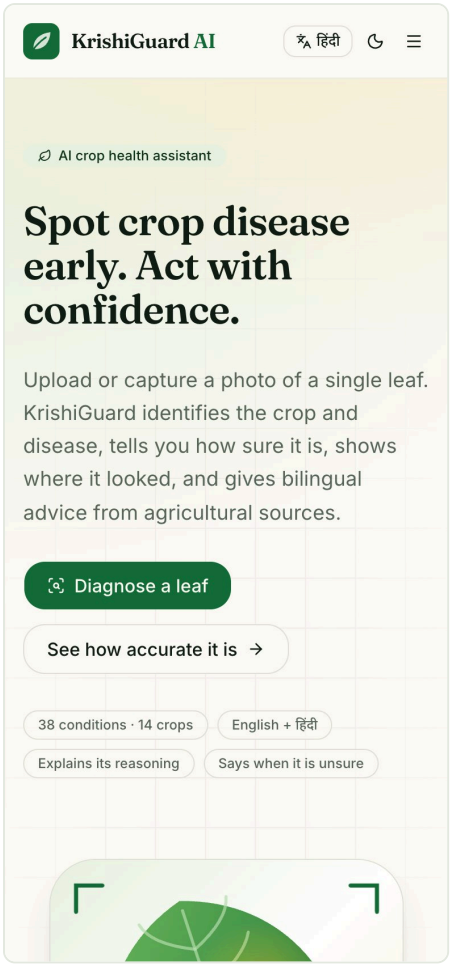


Figure 28. Mobile — landing.

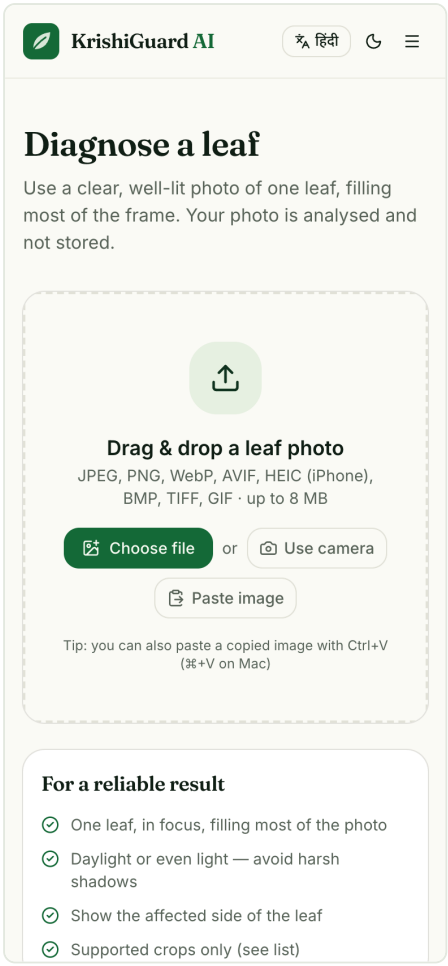


Figure 29. Mobile — diagnose.

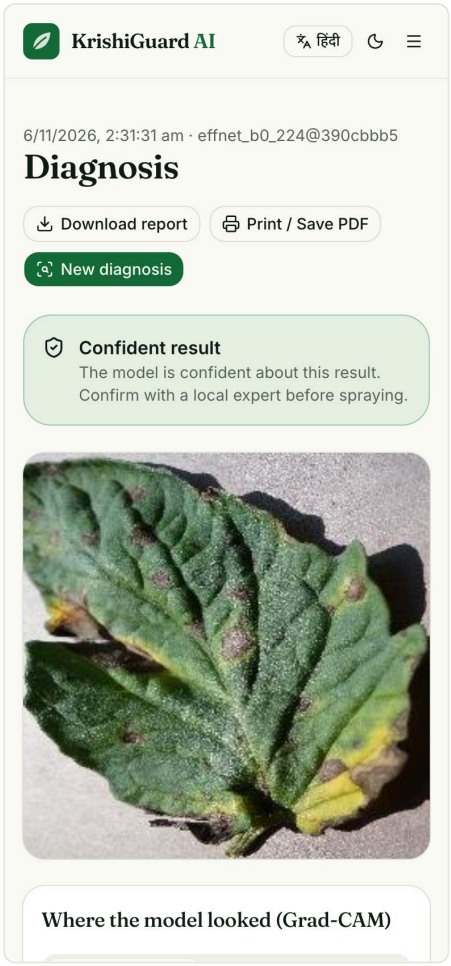


Figure 30. Mobile — result.

Other features: downloadable HTML diagnosis report, print / save-as-PDF, dark mode, feedback (stored as unverified), uncertainty warnings, blurry-photo warnings.

CHAPTER 13

Backend API and Model Serving

Endpoint	Purpose
GET /api/health	model / database / advisory status
GET /api/model/info	architecture, classes, temperature, thresholds, checksum
POST /api/predict	diagnose one photo (lang = enlhi, optional heat-map)
POST /api/predict/batch	diagnose up to 8 photos; per-file errors
GET /api/diseases, /api/diseases/{id}	disease list and detail (EN/HI)
GET /api/advisory/{id}	advisory for one disease in the chosen language
GET /api/metrics	evaluation summary for the dashboard (503 if not generated — never synthetic)
POST /api/feedback	user feedback, stored as unverified

- **Validation:** 8 MB limit, MIME check plus content sniffing, decompression-bomb guard, EXIF orientation, minimum size, blur detection; consistent JSON errors (413, 415, 422, 429, 503).
- **Serving:** the PyTorch model is exported to **ONNX** (16 MB) and run with ONNX Runtime; the deployment needs no PyTorch (≈ 133 MB of dependencies instead of > 1 GB) and inference takes about **35 ms** per image on the server (measured), versus 133 ms for the PyTorch model on the development laptop CPU.
- **Integrity:** the API refuses to load a model whose SHA-256 differs from the model card.
- **Privacy:** uploaded images are never stored — only a hash and prediction metadata.

CHAPTER 14

Testing and Quality Assurance

Suite	Tests	What is covered	Result
Backend (pytest)	44	preprocessing parity with training, 8 image formats, invalid uploads, ONNX loading & checksum, CAM = Grad-CAM, API endpoints, feedback, real-model integration	**all passed**
Frontend components (Vitest)	6	i18n switching, advisory rendering, heat-map viewer, report escaping	**all passed**
End-to-end (Playwright, Chrome)	16	desktop + mobile: full journey (upload → result → Hindi → heat-map → report → dashboard), library, unrecognised field photo, invalid upload, "New diagnosis" regression, AVIF, HEIC, paste	**all passed**
Live deployment	—	end-to-end suite and API checks run against the deployed Vercel site	**passed**

Bugs found and fixed during testing included a training hang on Apple MPS (replaced loss implementation), chart animation artefacts in print, a stale "Analysing…" state after *New diagnosis* (Next.js route state preservation) and a `.gitignore` rule that would have excluded the deployed model.

CHAPTER 15

Deployment

Both parts run on **Vercel's free (Hobby) plan** from the same GitHub repository; every `git push` redeploys them.

Project	Root	URL
Website (Next.js)	`frontend/`	krishiguard-ai-five.vercel.app
API (FastAPI, Python function)	repository root (`index.py`)	krishiguard-api.vercel.app/docs

The website proxies `/api/*` to the API project, so the browser talks to a single origin. Docker files and a docker-compose configuration (with PostgreSQL) are also provided for self-hosting.

CHAPTER 16

Limitations and Future Scope

- **Field accuracy:** 31.1%–26.5% on real field photos versus 99.6% on lab photos. *Future:* fine-tune on labelled field images (the PlantDoc domain-train split is already prepared) and collect Indian field data.
- **Closed set:** only 38 classes; other crops, pests or nutrient problems are forced onto these labels.
- **Single architecture:** ResNet-50 / Vision Transformer comparisons were planned but not trained, to keep the computational load suitable for a laptop; the training scripts support them.
- **Advisory review:** content is source-cited but has not yet been reviewed by an agronomist; pesticide registrations change.
- **Calibration:** one temperature cannot be right for both lab and field photos; confident field answers are still wrong about half the time.

CHAPTER 17

Conclusion

KrishiGuard AI demonstrates a complete, working pipeline from verified public datasets to a deployed, bilingual web application. A transfer-learned EfficientNet-B0 reaches **99.56% accuracy** and **99.49% macro-F1** on a leakage-free held-out test set, explains its decisions with heat-maps, and serves cited English and Hindi advice. Honest external testing shows that lab-trained models do not yet generalise to field photos; the application responds to this with calibrated confidence, explicit uncertainty states and by withholding advice when unsure.

References

1. Mohanty, S. P., Hughes, D. P., & Salathé, M. (2016). Using deep learning for image-based plant disease detection. *Frontiers in Plant Science*, 7, 1419.
2. Singh, D., Jain, N., Jain, P., Kayal, P., Kumawat, S., & Batra, N. (2020). PlantDoc: A dataset for visual plant disease detection. *CODS-COMAD 2020*.
3. Tomato Leaf Disease Classification Dataset in Pakistan. Mendeley Data, 3mbnb82mxd (CC BY 4.0).
4. Tan, M., & Le, Q. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. *ICML*.
5. Selvaraju, R. R., et al. (2017). Grad-CAM: Visual explanations from deep networks via gradient-based localization. *ICCV*.
6. Zhou, B., et al. (2016). Learning deep features for discriminative localization (CAM). *CVPR*.
7. Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. *ICML*.
8. Akiba, T., et al. (2019). Optuna: A next-generation hyperparameter optimization framework. *KDD*.
9. NIPHM / Directorate of Plant Protection, Quarantine & Storage — AESA-based IPM packages; TNAU Agritech Portal; and the other 50 advisory sources listed in `advisory/SOURCES.md`.

Appendix A — Per-class results

A.1 PlantVillage test set

Class	Precision	Recall	F1	Support
Apple — Apple scab	100.0%	100.0%	100.0%	88
Apple — Black rot	100.0%	100.0%	100.0%	91
Apple — Cedar apple rust	100.0%	100.0%	100.0%	39
Apple — healthy	100.0%	100.0%	100.0%	177
Blueberry — healthy	100.0%	100.0%	100.0%	136
Cherry — Powdery mildew	100.0%	100.0%	100.0%	150
Cherry — healthy	100.0%	100.0%	100.0%	128
Corn — Cercospora leaf spot Gray leaf spot	94.9%	97.4%	96.1%	76
Corn — Common rust	99.4%	99.4%	99.4%	178
Corn — Northern Leaf Blight	97.9%	96.6%	97.3%	147
Corn — healthy	100.0%	100.0%	100.0%	174
Grape — Black rot	98.9%	100.0%	99.4%	176
Grape — Esca (Black Measles)	100.0%	99.0%	99.5%	204
Grape — Leaf blight (Isariopsis Leaf Spot)	100.0%	100.0%	100.0%	160
Grape — healthy	100.0%	100.0%	100.0%	63
Orange — Haunglongbing (Citrus greening)	100.0%	100.0%	100.0%	822
Peach — Bacterial spot	100.0%	100.0%	100.0%	342
Peach — healthy	100.0%	100.0%	100.0%	54
Pepper (bell) — Bacterial spot	99.3%	100.0%	99.7%	147
Pepper (bell) — healthy	100.0%	99.5%	99.8%	218
Potato — Early blight	100.0%	98.0%	99.0%	148
Potato — Late blight	99.3%	99.3%	99.3%	148
Potato — healthy	100.0%	100.0%	100.0%	16
Raspberry — healthy	100.0%	100.0%	100.0%	38
Soybean — healthy	100.0%	100.0%	100.0%	633
Squash — Powdery mildew	100.0%	100.0%	100.0%	275
Strawberry — Leaf scorch	100.0%	100.0%	100.0%	162
Strawberry — healthy	100.0%	100.0%	100.0%	60
Tomato — Bacterial spot	99.4%	99.4%	99.4%	316
Tomato — Early blight	95.4%	98.0%	96.7%	148
Tomato — Late blight	98.6%	97.9%	98.2%	283
Tomato — Leaf Mold	100.0%	98.6%	99.3%	140
Tomato — Septoria leaf spot	100.0%	99.6%	99.8%	265
Tomato — Spider mites Two-spotted spider mite	98.4%	99.6%	99.0%	248
Tomato — Target Spot	98.6%	100.0%	99.3%	210
Tomato — Tomato Yellow Leaf Curl Virus	99.9%	99.5%	99.7%	801
Tomato — Tomato mosaic virus	100.0%	100.0%	100.0%	55

Class	Precision	Recall	F1	Support
Tomato — healthy	99.6%	100.0%	99.8%	235

A.2 External field test sets

Set	Class	Precision	Recall	F1	Support
PlantDoc	Apple — Apple scab	100.0%	20.0%	33.3%	10
PlantDoc	Apple — Cedar apple rust	0.0%	0.0%	0.0%	10
PlantDoc	Apple — healthy	30.0%	33.3%	31.6%	9
PlantDoc	Blueberry — healthy	33.3%	50.0%	40.0%	10
PlantDoc	Cherry — healthy	33.3%	20.0%	25.0%	10
PlantDoc	Corn — Cercospora leaf spot Gray leaf spot	13.3%	66.7%	22.2%	3
PlantDoc	Corn — Common rust	66.7%	20.0%	30.8%	10
PlantDoc	Corn — Northern Leaf Blight	42.9%	30.0%	35.3%	10
PlantDoc	Grape — Black rot	44.4%	50.0%	47.1%	8
PlantDoc	Grape — healthy	60.0%	75.0%	66.7%	12
PlantDoc	Peach — healthy	100.0%	11.1%	20.0%	9
PlantDoc	Pepper (bell) — Bacterial spot	66.7%	44.4%	53.3%	9
PlantDoc	Pepper (bell) — healthy	50.0%	75.0%	60.0%	8
PlantDoc	Potato — Early blight	40.0%	33.3%	36.4%	6
PlantDoc	Potato — Late blight	0.0%	0.0%	0.0%	5
PlantDoc	Raspberry — healthy	55.6%	71.4%	62.5%	7
PlantDoc	Soybean — healthy	0.0%	0.0%	0.0%	8
PlantDoc	Squash — Powdery mildew	50.0%	66.7%	57.1%	6
PlantDoc	Strawberry — healthy	0.0%	0.0%	0.0%	8
PlantDoc	Tomato — Bacterial spot	0.0%	0.0%	0.0%	7
PlantDoc	Tomato — Early blight	28.0%	77.8%	41.2%	9
PlantDoc	Tomato — Late blight	12.5%	33.3%	18.2%	9
PlantDoc	Tomato — Leaf Mold	0.0%	0.0%	0.0%	6
PlantDoc	Tomato — Septoria leaf spot	25.0%	30.0%	27.3%	10
PlantDoc	Tomato — Tomato Yellow Leaf Curl Virus	100.0%	40.0%	57.1%	5
PlantDoc	Tomato — Tomato mosaic virus	0.0%	0.0%	0.0%	10
PlantDoc	Tomato — healthy	0.0%	0.0%	0.0%	8
Tomato field	Tomato — Early blight	31.7%	55.9%	40.4%	1200
Tomato field	Tomato — Late blight	35.0%	67.2%	46.0%	1200
Tomato field	Tomato — Leaf Mold	96.4%	4.4%	8.4%	1200
Tomato field	Tomato — Septoria leaf spot	49.6%	11.1%	18.1%	1200
Tomato field	Tomato — Tomato Yellow Leaf Curl Virus	79.0%	20.3%	32.3%	1200
Tomato field	Tomato — healthy	0.0%	0.0%	0.0%	1200

Appendix B — Dataset split per class

Class	Train	Validation	Test	PlantDoc ext-val	External test
Apple — Apple scab	441	101	88	16	10
Apple — Black rot	449	81	91	0	0
Apple — Cedar apple rust	196	40	39	16	10
Apple — healthy	1305	163	177	16	9
Blueberry — healthy	1246	120	136	21	10
Cherry — Powdery mildew	745	157	150	0	0
Cherry — healthy	603	123	128	10	10
Corn — Cercospora leaf spot Gray leaf spot	360	77	76	13	3
Corn — Common rust	835	179	178	21	10
Corn — Northern Leaf Blight	690	148	147	36	10
Corn — healthy	814	174	174	0	0
Grape — Black rot	828	176	176	11	8
Grape — Esca (Black Measles)	972	207	204	0	0
Grape — Leaf blight (Isariopsis Leaf Spot)	756	160	160	0	0
Grape — healthy	297	63	63	11	12
Orange — Haunglongbing (Citrus greening)	3865	820	822	0	0
Peach — Bacterial spot	1612	343	342	0	0
Peach — healthy	254	52	54	20	9
Pepper (bell) — Bacterial spot	699	151	147	13	9
Pepper (bell) — healthy	1036	224	218	11	8
Potato — Early blight	700	152	148	21	6
Potato — Late blight	700	152	148	18	5
Potato — healthy	112	24	16	0	0
Raspberry — healthy	265	68	38	23	7
Soybean — healthy	4389	68	633	12	8
Squash — Powdery mildew	1285	275	275	24	6
Strawberry — Leaf scorch	780	167	162	0	0
Strawberry — healthy	359	37	60	18	8
Tomato — Bacterial spot	1491	320	316	20	7
Tomato — Early blight	700	152	148	15	1209
Tomato — Late blight	1337	289	283	19	1209
Tomato — Leaf Mold	668	144	140	17	1206
Tomato — Septoria leaf spot	1240	266	265	26	1210
Tomato — Spider mites Two-spotted spider mite	1176	252	248	1	0
Tomato — Target Spot	983	211	210	0	0
Tomato — Tomato Yellow Leaf Curl Virus	3752	804	801	14	1205
Tomato — Tomato mosaic virus	262	56	55	9	10
Tomato — healthy	1107	249	235	11	1208